



Борис Цейтлин

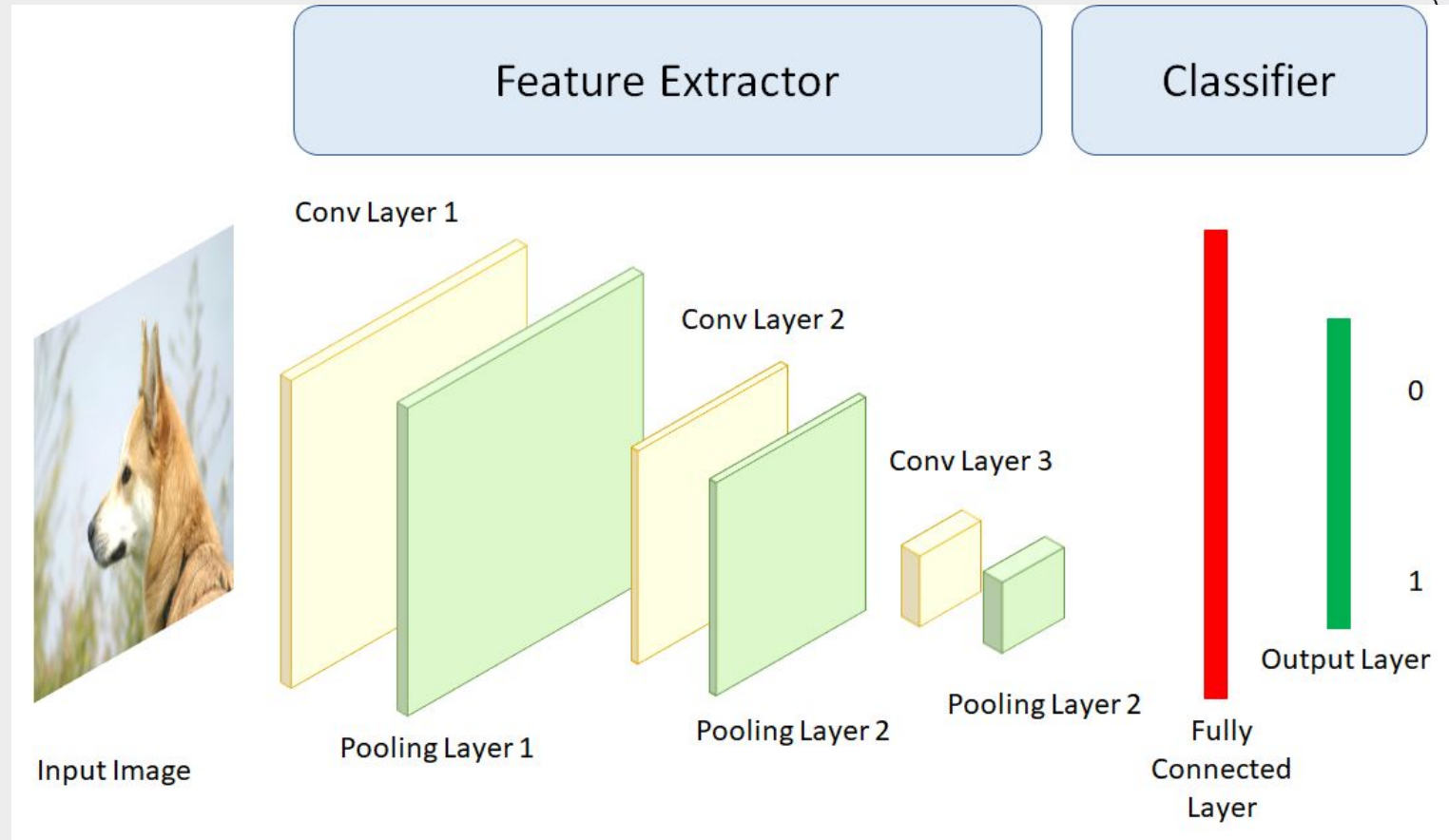
Админ @boris_again, ex-Staff MLE eBay

Self-Supervised Computer Vision

Зачем нужен SSL?

Важность данных и
размеров моделей

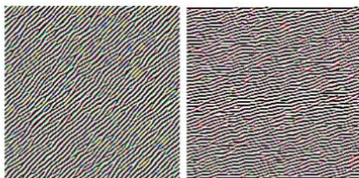
Pre-training + Finetuning



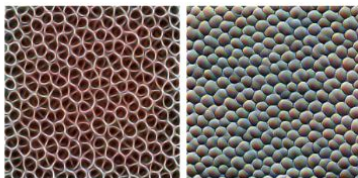
Универсальные внутренние репрезентации

Feature Visualization

How neural networks build up their understanding of images



Edges (layer conv2d0)



Textures (layer mixed3a)



Patterns (layer mixed4a)



Parts (layers mixed4b & mixed4c)



Objects (layers mixed4d & mixed4e)

Использование backbone нейросети для разных задач

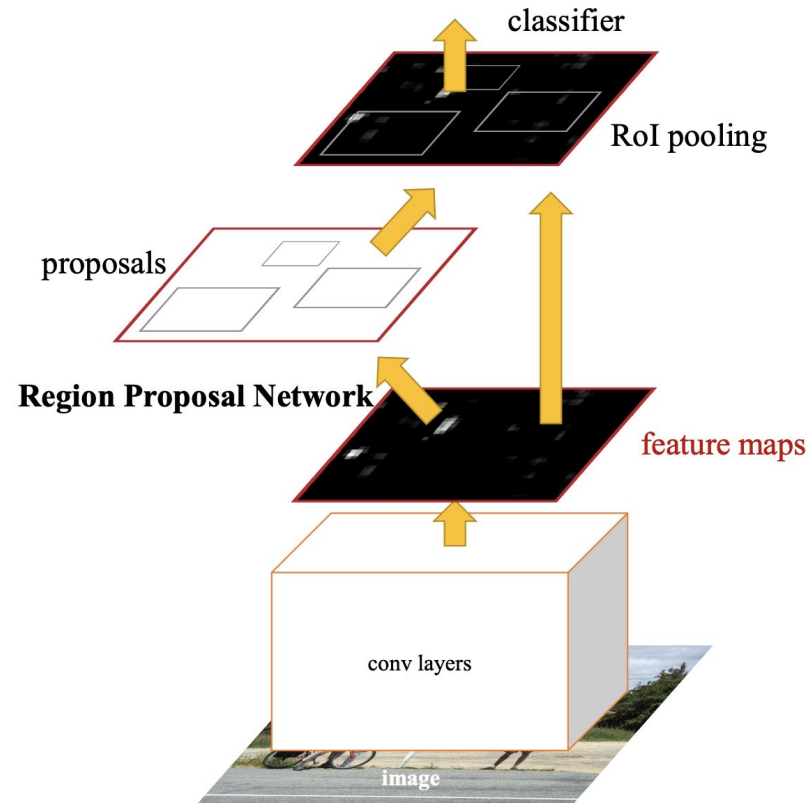
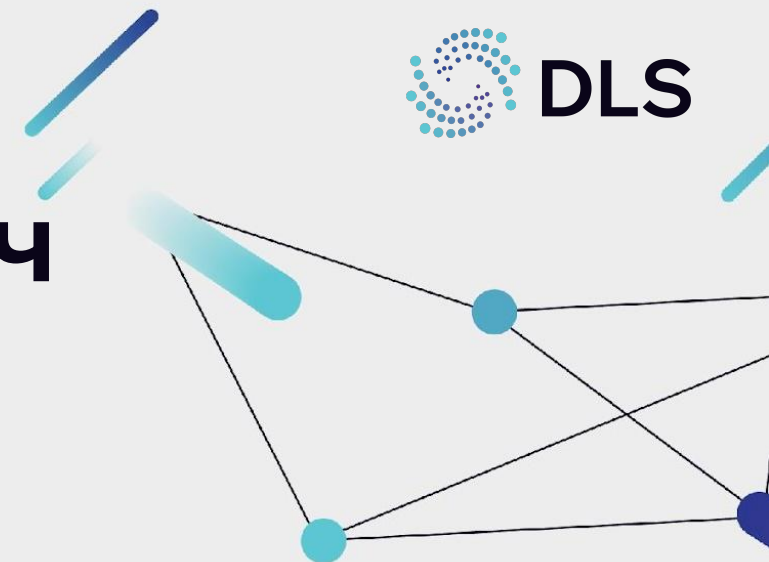
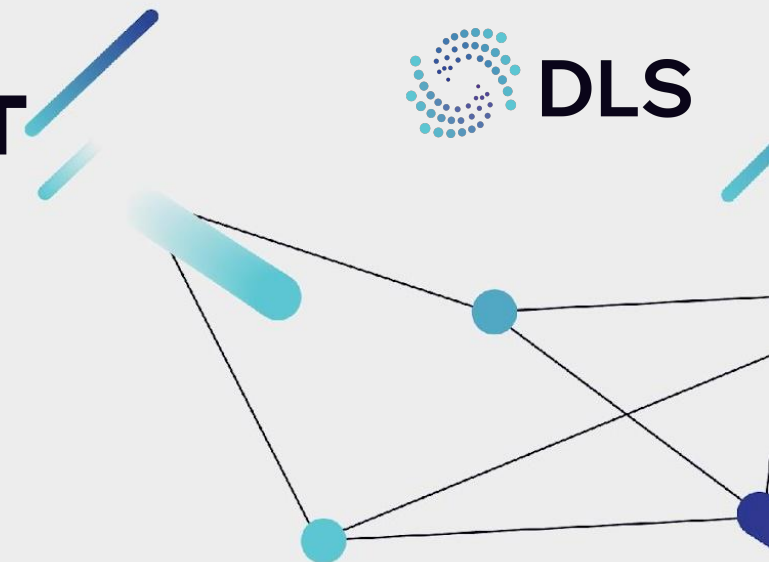


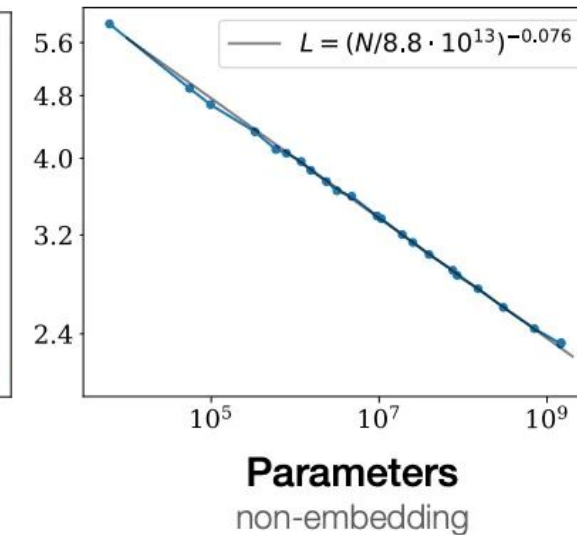
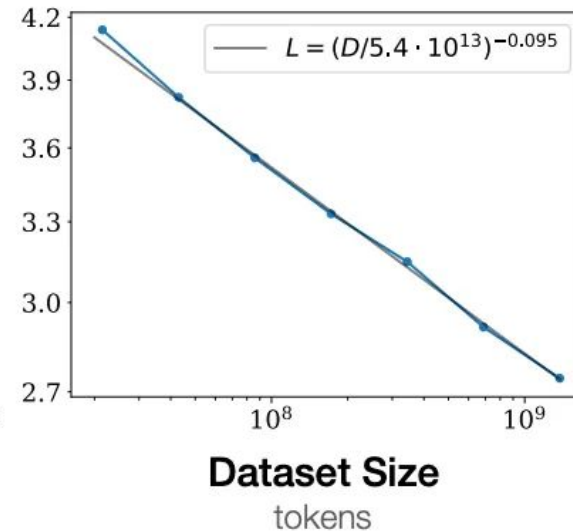
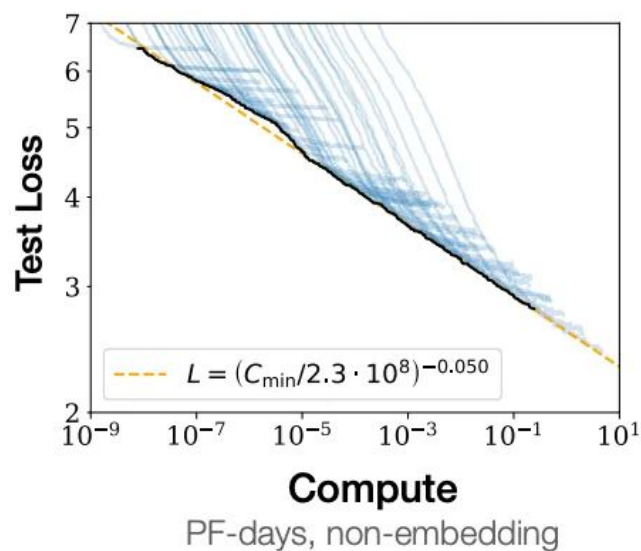
Figure 2: Faster R-CNN is a single, unified network for object detection. The RPN module serves as the 'attention' of this unified network.



Supervised подходы делают глупенькие модели



Что у соседей в NLP: Scaling Laws



Scaling Laws Дома : (

Image Classification on ImageNet

Leaderboard

Dataset

View

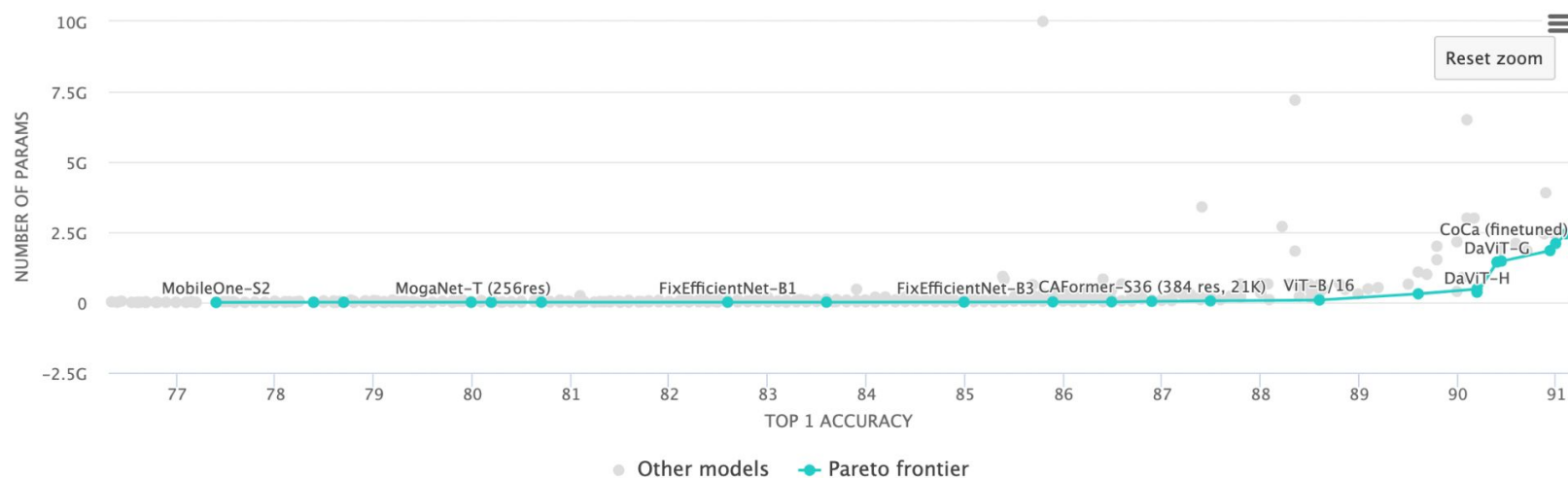
Number of params

by

Top 1 Accuracy

for

All models



Сравнение датасетов

The Pile: 800 GB

Imagenet 21k: 14m примеров, ~150 GB

YFCC100M: 100m примеров, ~15, качественных

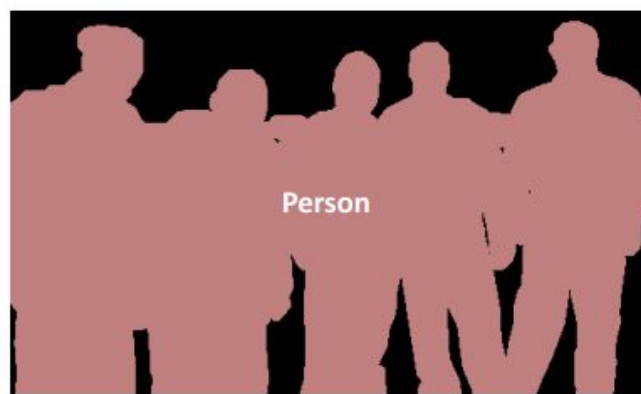
MS-COCO: 100k примеров



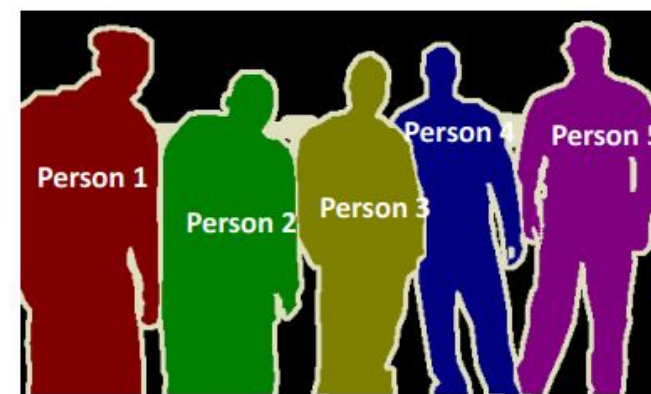
Разметка в CV дорогая



Object Detection

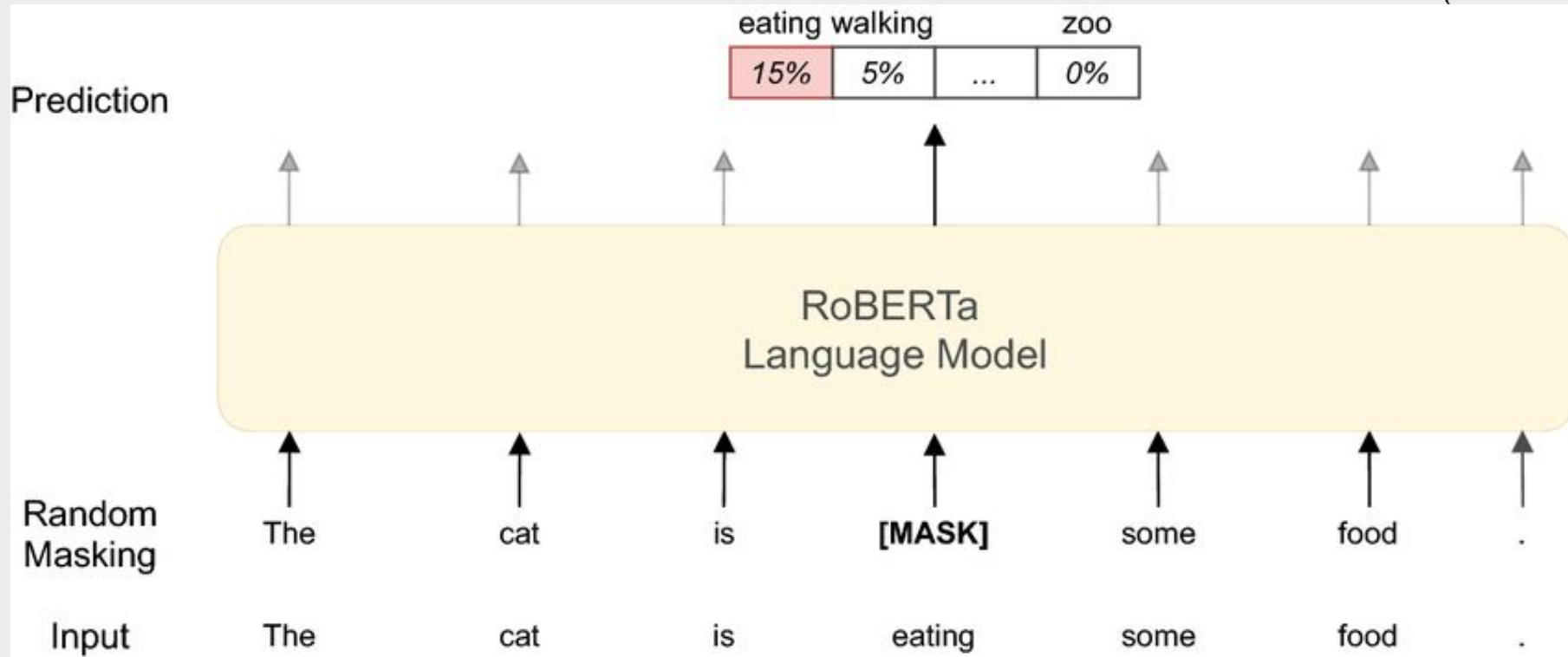


Semantic Segmentation



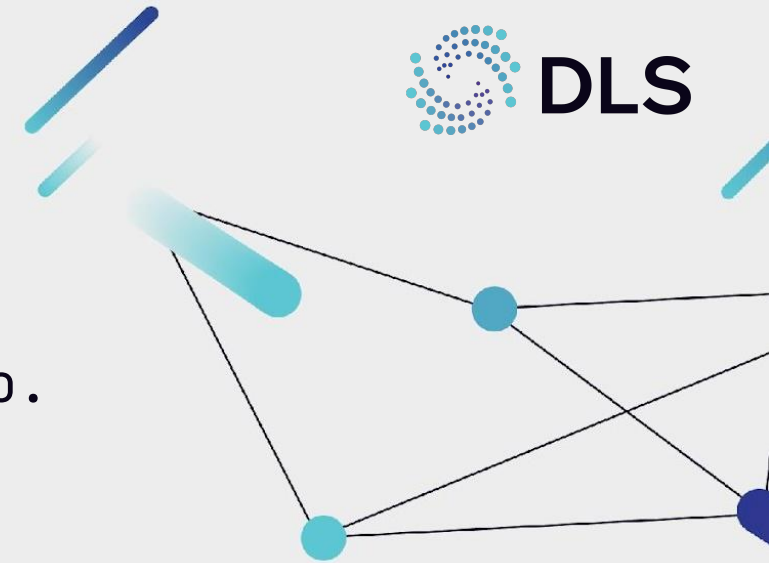
Instance Segmentation

Почему в NLP все так масштабно? Self-Supervised Learning



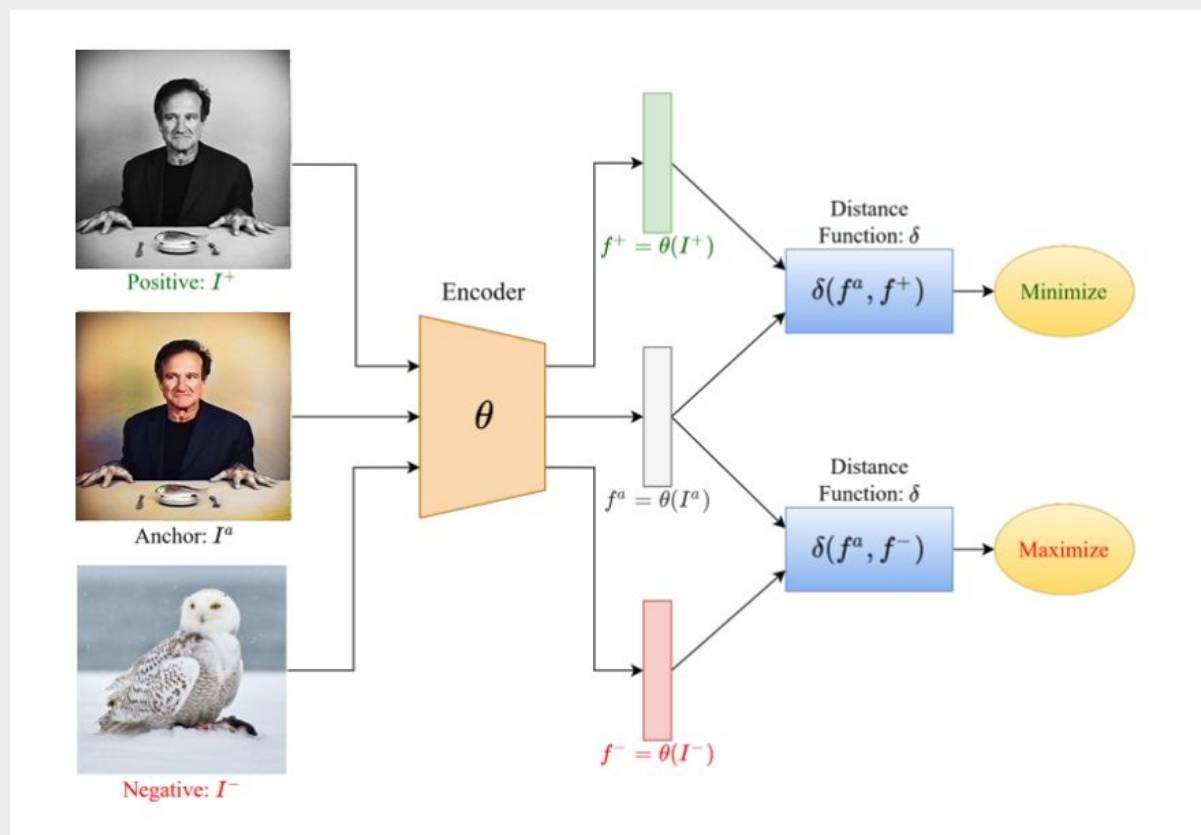
Ключевые тезисы

1. Больше данных $\Rightarrow$ больше модели $\Rightarrow$ лучше качество.
2. Изображений достаточно мало.
3. Разметки для изображений еще меньше и она очень дорогая.
4. Если бы могли использовать больше неразмеченных данных, то получили бы более крутые модели.



Классические SSL методы в CV: SimCLR, BYOL

Contrastive Learning



A Simple Framework for Contrastive Learning of Visual Representations

Ting Chen¹ Simon Kornblith¹ Mohammad Norouzi¹ Geoffrey Hinton¹

<https://arxiv.org/pdf/2002.05709>

SimCLR: key result

Можно получить качество на уровне supervised обучения используя в 100 раз меньше лейблов.

1. Предобучаем с помощью SSL метода.
2. Дообучаем на небольшом количестве примеров с лейблами.

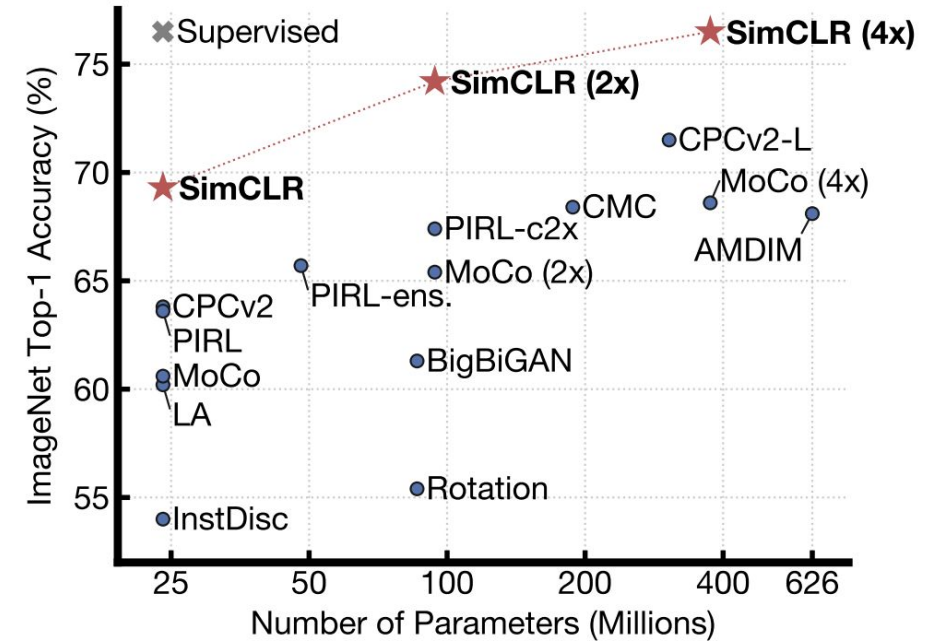
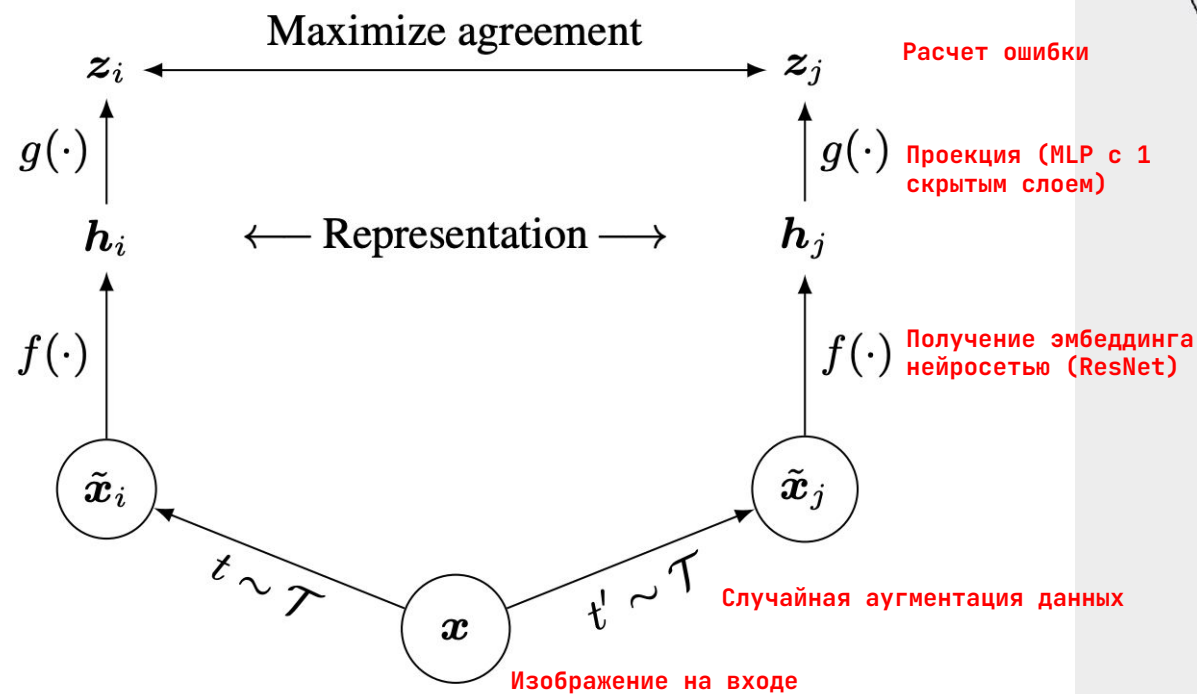


Figure 1. ImageNet Top-1 accuracy of linear classifiers trained on representations learned with different self-supervised methods (pretrained on ImageNet). Gray cross indicates supervised ResNet-50. Our method, SimCLR, is shown in bold.

SimCLR: метод



SimCLR: аугментации

Лучший набор аугментаций:

1. Random Resized Crop
2. Random Color Distortion
 - a. With random grayscaling
3. Random Gaussian Blur

Выводы:

1. Выбор аугментаций очень важен.
2. Random Crop создает хорошую и универсальную задачу.

A Simple Framework for Contrastive Learning of Visual Representations

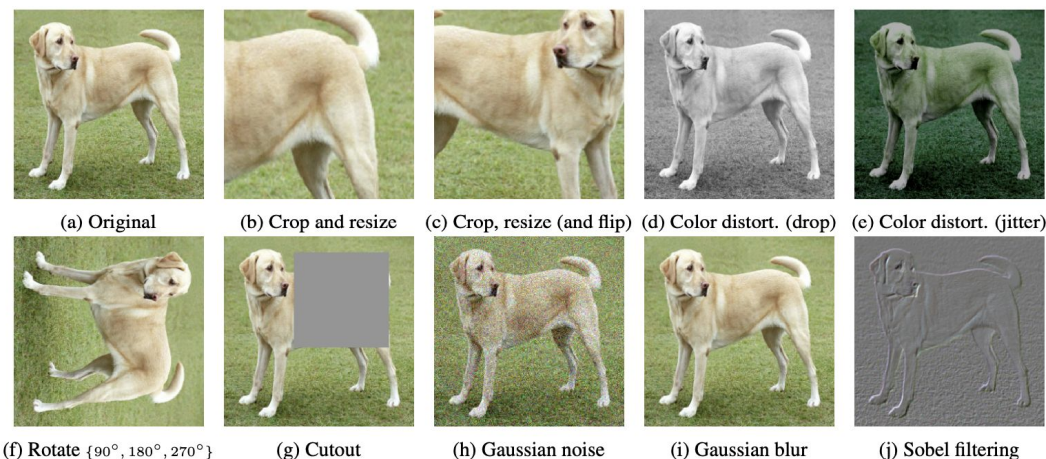


Figure 4. Illustrations of the studied data augmentation operators. Each augmentation can transform data stochastically with some internal parameters (e.g. rotation degree, noise level). Note that we *only* test these operators in ablation, the *augmentation policy used to train our models* only includes *random crop (with flip and resize)*, *color distortion*, and *Gaussian blur*. (Original image cc-by: Von.grzanka)

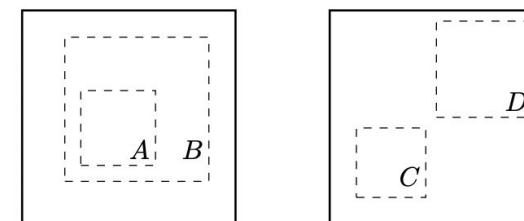


Figure 3. Solid rectangles are images, dashed rectangles are random crops. By randomly cropping images, we sample contrastive prediction tasks that include global to local view ($B \rightarrow A$) or adjacent view ($D \rightarrow C$) prediction.

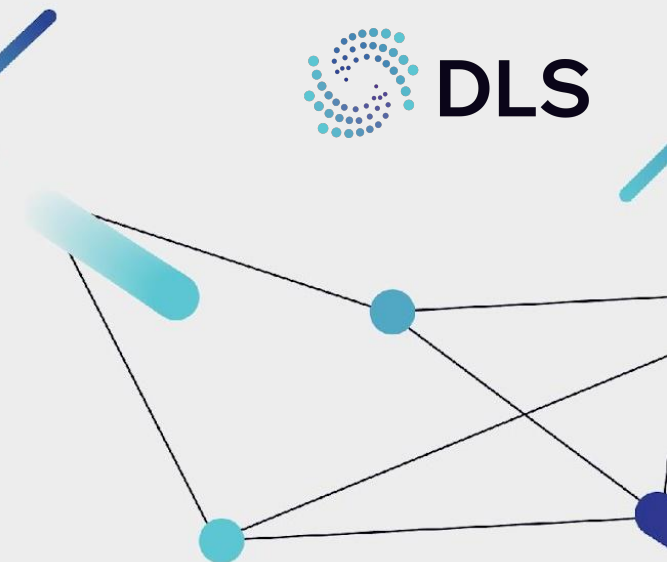
SimCLR: алгоритм обучения

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N , constant τ , structure of f , g , $\mathcal{T}$.
for sampled minibatch $\{\mathbf{x}_k\}_{k=1}^N$ **do**
 for all $k \in \{1, \dots, N\}$ **do**
 draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
 # the first augmentation
 $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$
 $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$ # representation
 $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$ # projection
 # the second augmentation
 $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$
 $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$ # representation
 $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$ # projection
 end for
 for all $i \in \{1, \dots, 2N\}$ and $j \in \{1, \dots, 2N\}$ **do**
 $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$ # pairwise similarity
 end for
 define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
 $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
 update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

Эмбединг который мы на самом деле обучаем

Эмбединг после проекции - нужен только для обучения



SimCLR: NT-Xent/InfoNCE loss

По сути мы создаем себе **задачу классификации**.

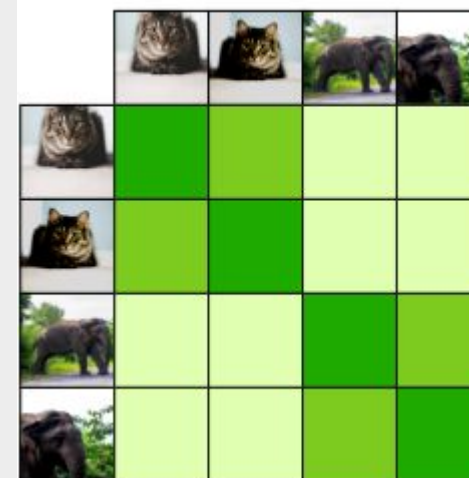
1. Семплируем N изображений в батч.
2. С помощью аугментаций получаем 2N примеров.
3. Всем парам представлений полученным из одного изображения ставим класс 1, всем остальным парам класс 0.
4. “логитом” считаем скалярное произведение между представлениями.
5. Cross-entropy loss.

<https://amitness.com/posts/simclr>

```

for all  $i \in \{1, \dots, 2N\}$  and  $j \in \{1, \dots, 2N\}$  do
     $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$       # pairwise similarity
end for
define  $\ell(i, j)$  as  $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$ 
 $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$ 
update networks  $f$  and  $g$  to minimize  $\mathcal{L}$ 
    
```

Pairwise cosine similarity



SimCLR: NT-Xent/InfoNCE loss

```
def info_nce_loss(self, features):  
  
    labels = torch.cat([torch.arange(self.args.batch_size) for i in range(self.args.n_views)], dim=0)  
    labels = (labels.unsqueeze(0) == labels.unsqueeze(1)).float()  
    labels = labels.to(self.args.device)  
  
    features = F.normalize(features, dim=1)  
  
    similarity_matrix = torch.matmul(features, features.T)  
    # assert similarity_matrix.shape == (  
    #     self.args.n_views * self.args.batch_size, self.args.n_views * self.args.batch_size)  
    # assert similarity_matrix.shape == labels.shape  
  
    # discard the main diagonal from both: labels and similarities matrix  
    mask = torch.eye(labels.shape[0], dtype=torch.bool).to(self.args.device)  
    labels = labels[~mask].view(labels.shape[0], -1)  
    similarity_matrix = similarity_matrix[~mask].view(similarity_matrix.shape[0], -1)  
    # assert similarity_matrix.shape == labels.shape  
  
    # select and combine multiple positives  
    positives = similarity_matrix[labels.bool()].view(labels.shape[0], -1)  
  
    # select only the negatives the negatives  
    negatives = similarity_matrix[~labels.bool()].view(similarity_matrix.shape[0], -1)  
  
    logits = torch.cat([positives, negatives], dim=1)  
    labels = torch.zeros(logits.shape[0], dtype=torch.long).to(self.args.device)  
  
    logits = logits / self.args.temperature  
    return logits, labels
```

[https://github.com/sthalles/SimCLR/
blob/master/simclr.py#L26](https://github.com/sthalles/SimCLR/blob/master/simclr.py#L26)

SimCLR: Проекция имеет значение

Добавление MLP в конце, который выбрасывается после обучения, улучшает качество эмбеддингов.

$g(h)$ тренируется быть инвариантным относительно трансформаций над данными. Но эта информация может быть нам полезна!

h может содержать больше информации (полезной для downstream задач) если не используется напрямую для расчета лосса.

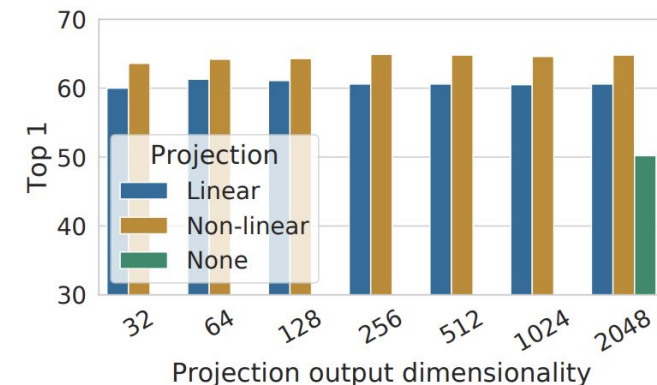


Figure 8. Linear evaluation of representations with different projection heads $g(\cdot)$ and various dimensions of $z = g(h)$. The representation h (before projection) is 2048-dimensional here.

What to predict?	Random guess	Representation	
		h	$g(h)$
Color vs grayscale	80	99.3	97.4
Rotation	25	67.6	25.6
Orig. vs corrupted	50	99.5	59.6
Orig. vs Sobel filtered	50	96.6	56.3

Table 3. Accuracy of training additional MLPs on different representations to predict the transformation applied. Other than crop and color augmentation, we additionally and independently add rotation (one of $\{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$), Gaussian noise, and Sobel filtering transformation during the pretraining for the last three rows. Both h and $g(h)$ are of the same dimensionality, i.e. 2048.

SimCLR: Большие батчсайзы!

Imagenet

CIFAR-10

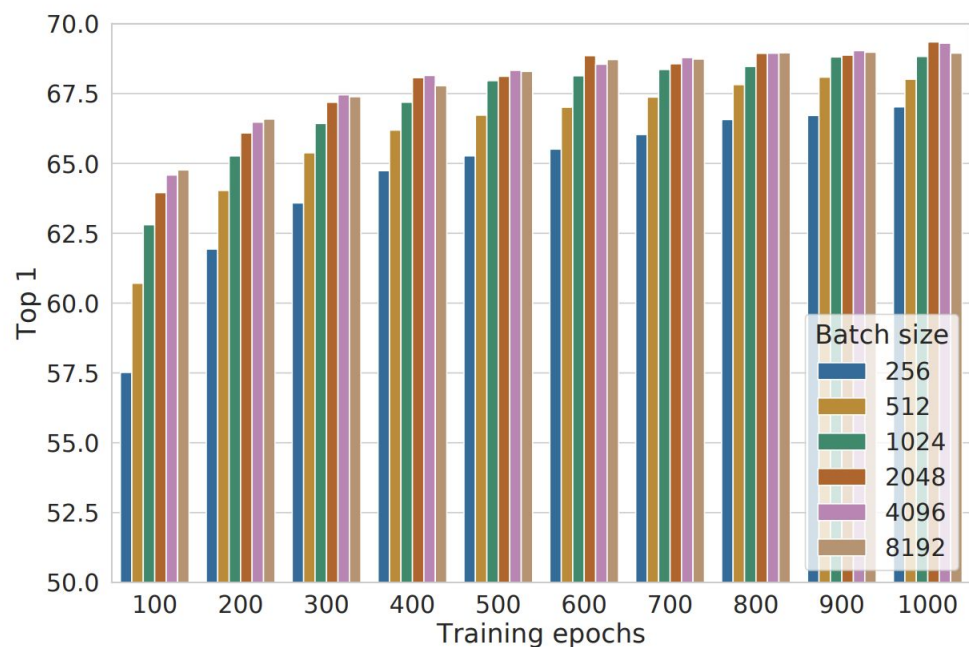
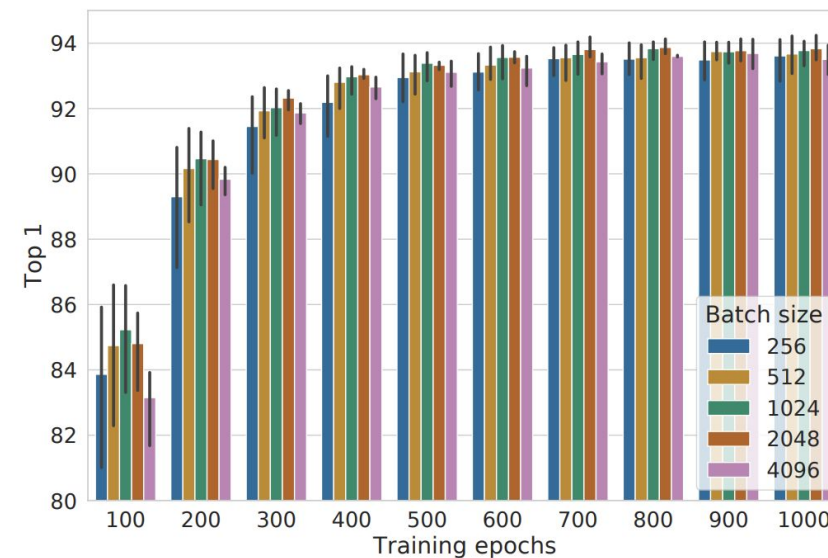


Figure 9. Linear evaluation models (ResNet-50) trained with different batch size and epochs. Each bar is a single run from scratch.¹⁰



SimCLR: результаты

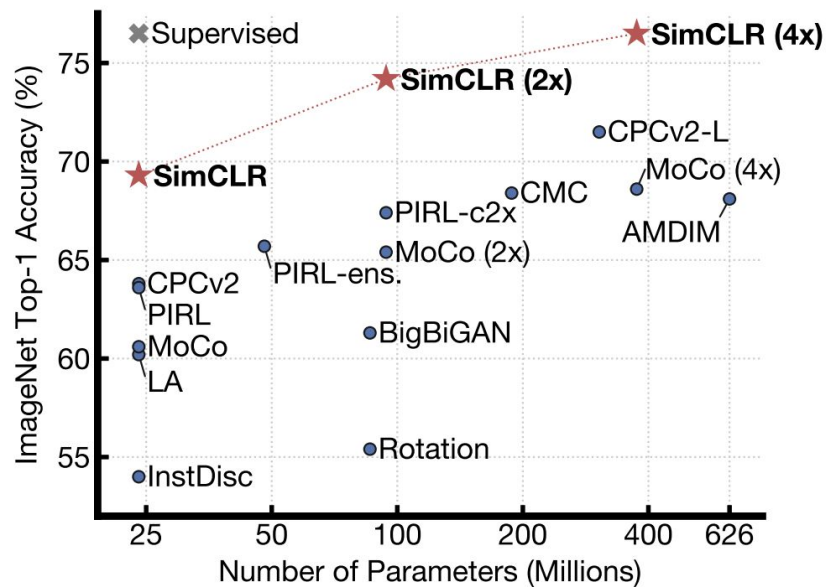


Figure 1. ImageNet Top-1 accuracy of linear classifiers trained on representations learned with different self-supervised methods (pretrained on ImageNet). Gray cross indicates supervised ResNet-50. Our method, SimCLR, is shown in bold.

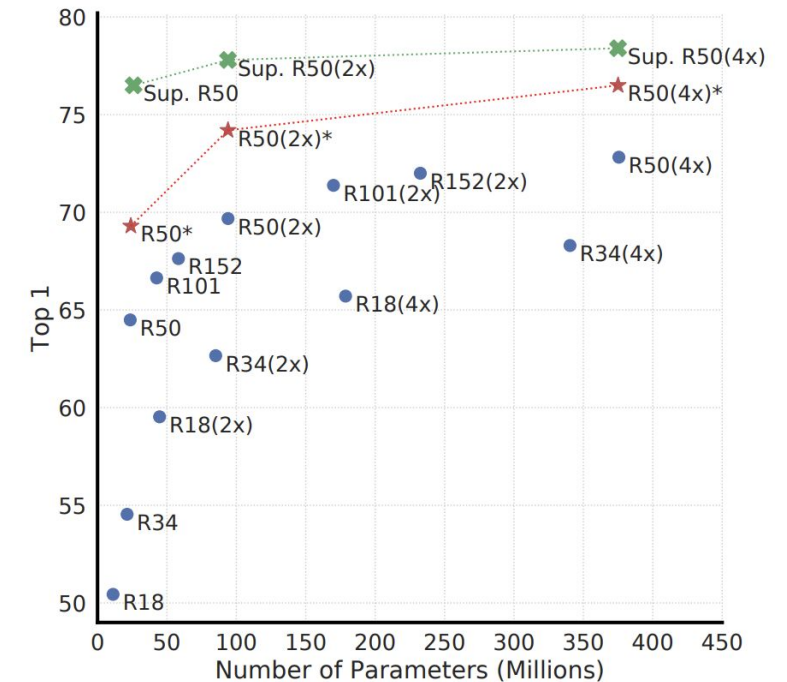


Figure 7. Linear evaluation of models with varied depth and width. Models in blue dots are ours trained for 100 epochs, models in red stars are ours trained for 1000 epochs, and models in green crosses are supervised ResNets trained for 90 epochs⁷ (He et al., 2016).

Проблемы SimCLR

- **Очень большой batch size:** 4096+
 - Который потом увеличивается в два раза.
- Нужно долго обучать.
 - Вероятная причина: false negatives в батчах.
- Нужны большие модели, но тяжело использовать большие backbone из-за ограничений памяти.
- Нужно **очень** много данных.
- Чувствительность к аугментациям.



Bootstrap Your Own Latent

A New Approach to Self-Supervised Learning

Jean-Bastien Grill^{*,1} Florian Strub^{*,1} Florent Altché^{*,1} Corentin Tallec^{*,1} Pierre H. Richemond^{*,1,2}

Elena Buchatskaya¹ Carl Doersch¹ Bernardo Avila Pires¹ Zhaohan Daniel Guo¹

Mohammad Gheshlaghi Azar¹ Bilal Piot¹ Koray Kavukcuoglu¹ Rémi Munos¹ Michal Valko¹

¹DeepMind

²Imperial College

[jbgrill,fstrub,altche,corentint,richemond]@google.com

<https://arxiv.org/pdf/2006.07733>

BYOL: key result

SSL может быть гораздо эффективнее и стабильнее!

- Качество лучше, чем у SimCLR, при размере батча около в 10-20 раз меньше (256 vs 4096).
- Меньшая чувствительность к выбору аугментаций

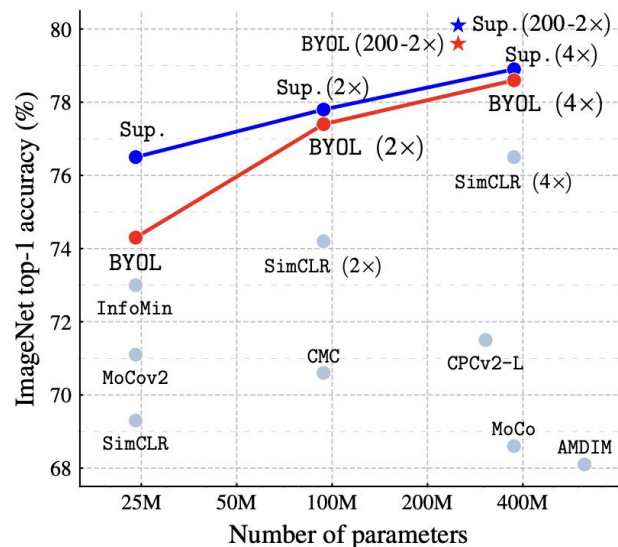
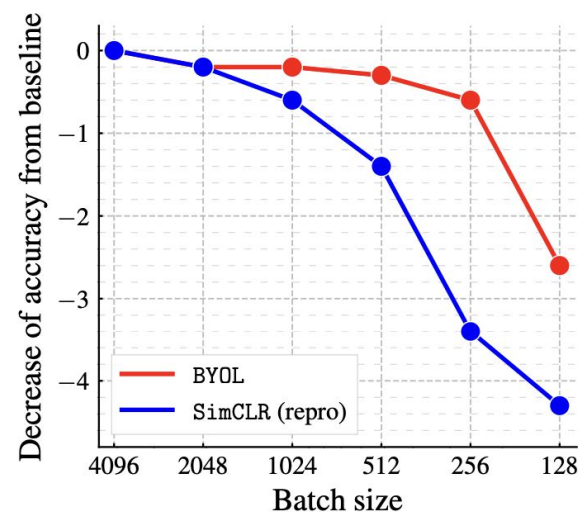
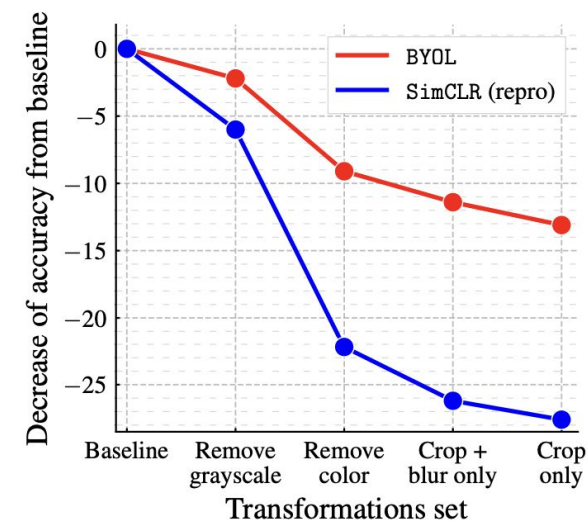


Figure 1: Performance of BYOL on ImageNet (linear evaluation) using ResNet-50 and our best architecture ResNet-200 (2x), compared to other unsupervised and supervised (Sup.) baselines [8].



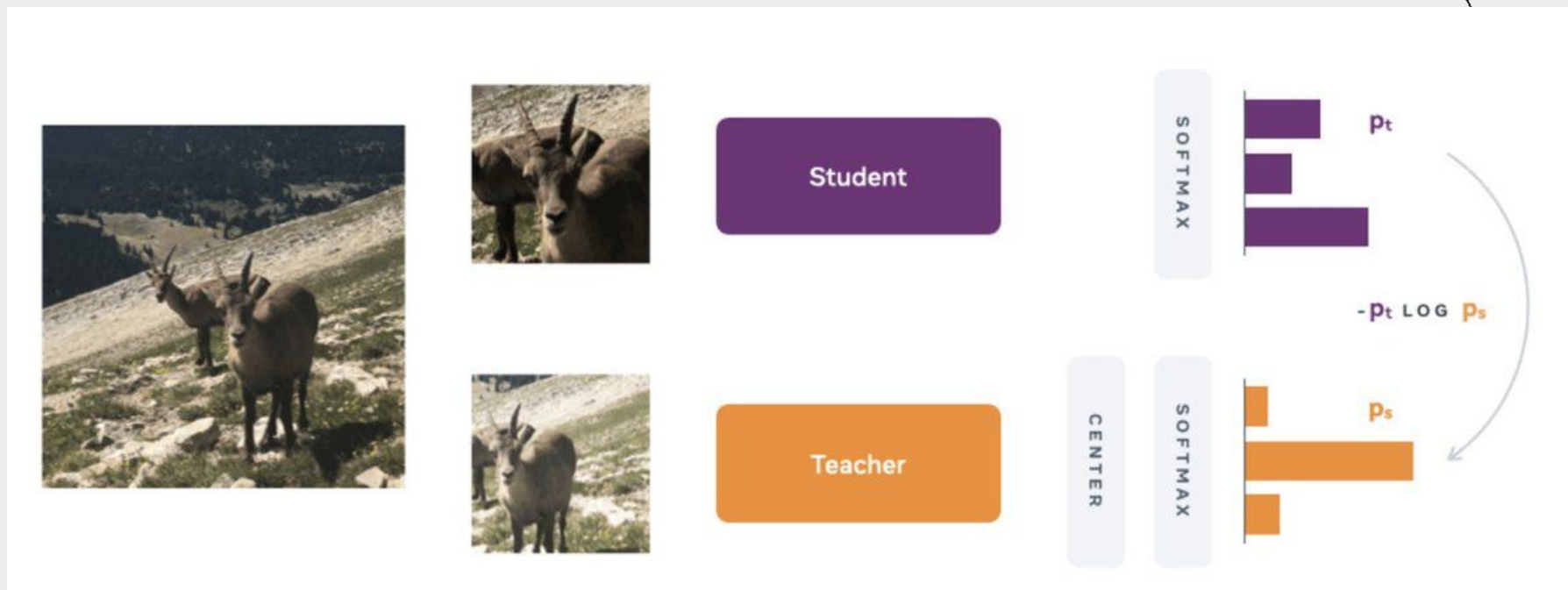
(a) Impact of batch size



(b) Impact of progressively removing transformations

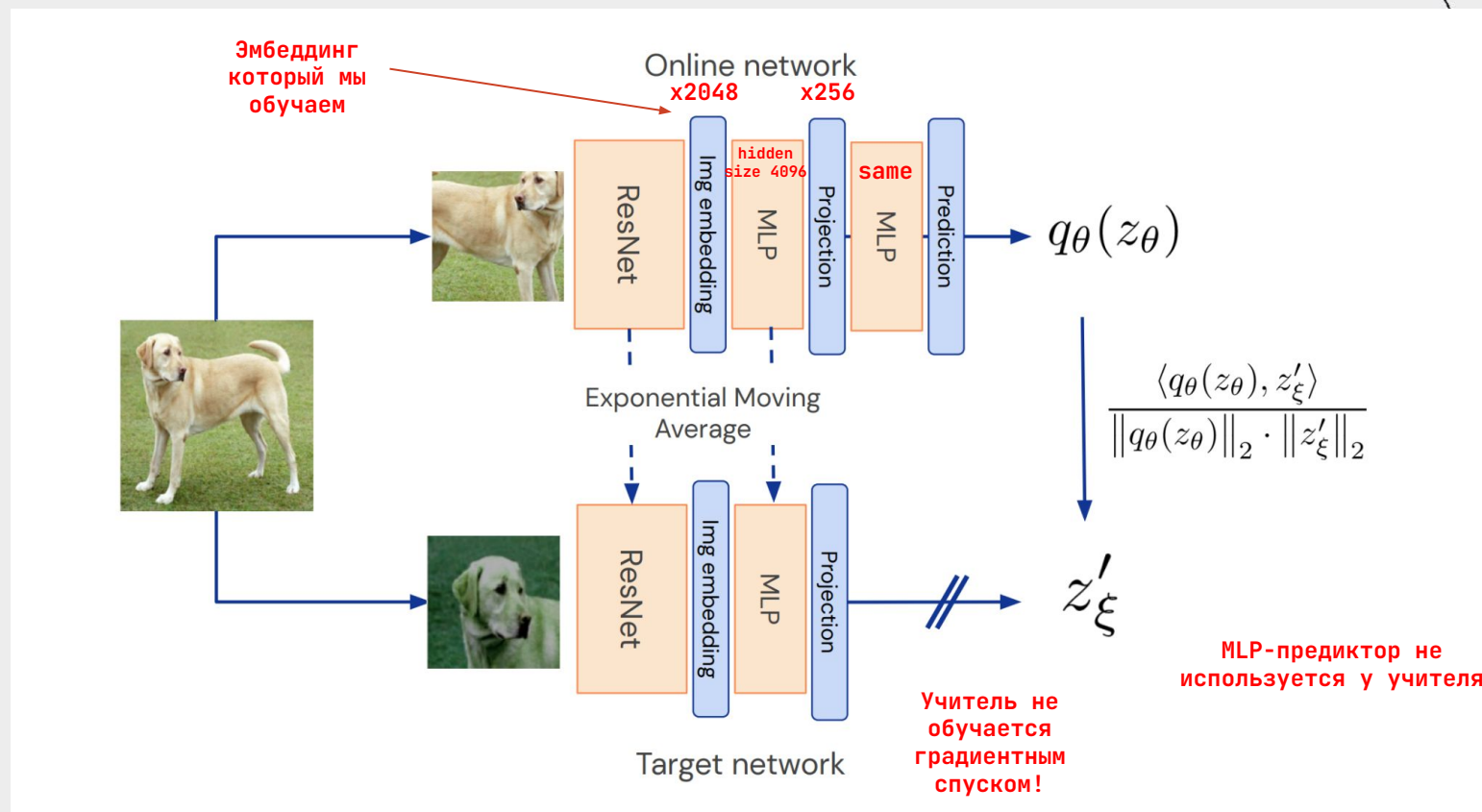
Figure 3: Decrease in top-1 accuracy (in % points) of BYOL and our own reproduction of SimCLR at 300 epochs, under linear evaluation on ImageNet.

Вспоминаем дистилляцию



https://docs.pytorch.org/tutorials/beginner/knowledge_distillation_tutorial.html#knowledge-distillation-run

BYOL: асимметричная архитектура



Считаем ошибку на основе разницы между предсказаниями учителя и ученика

BYOL: ошибка и обучение

$$\mathcal{L}_{\theta, \xi} \triangleq \left\| \overline{q_{\theta}}(z_{\theta}) - \overline{z'_{\xi}} \right\|_2^2 = 2 - 2 \cdot \frac{\langle q_{\theta}(z_{\theta}), z'_{\xi} \rangle}{\|q_{\theta}(z_{\theta})\|_2 \cdot \|z'_{\xi}\|_2}.$$

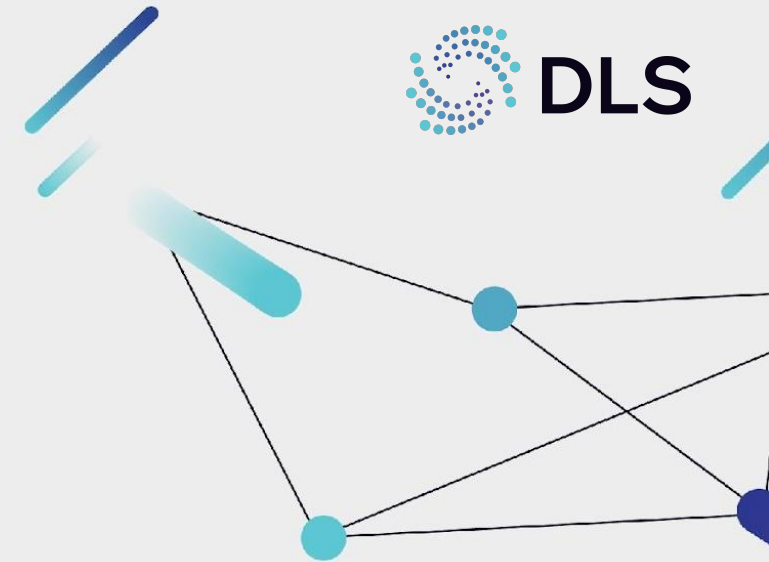
Предикт
ученика

Проекция
учителя

$$\begin{aligned} \theta &\leftarrow \text{optimizer}(\theta, \nabla_{\theta} \mathcal{L}_{\theta, \xi}^{\text{BYOL}}, \eta), \\ \xi &\leftarrow \tau \xi + (1 - \tau) \theta, \end{aligned}$$

BYOL: погодите, что?

- Модель учится сама у себя?
 - Что за магия?
 - Разве они не сойдутся в одну точку с ошибкой 0?
- Нет негативных примеров?
 - Почему оно не коллапсирует в константу?



BYOL: становится интереснее

Пояснения:

- $t_{\text{base}} = 1$ -> учитель не обучается
- $t_{\text{base}} = 0$ -> учитель равен ученику на предыдущем шаге

Вывод:

Даже если учиться на таргетах случайной модели качество вырастает до 18.8% (качество самой случайной модели примерно 1%).

Target	τ_{base}	Top-1
Constant random network	1	18.8 $\pm$ 0.7
Moving average of online	0.999	69.8
Moving average of online	0.99	72.5
Moving average of online	0.9	68.4
Stop gradient of online [†]	0	0.3

(a) Results for different target modes. [†]In the *stop gradient of online*, $\tau = \tau_{\text{base}} = 0$ is kept constant throughout training.

BYOL: как модель может учиться сама у себя?



1. Предсказания ученика всегда отличаются из-за наличия MLP-предиктора.
2. Учитель со скользящим средним дает более стабильные предсказания.

Главное:

Обновление учителя происходит **в сторону весов ученика, а не в сторону градиента ученика.**

Доказательство арифметикой.

Пусть веса ученика [1, 2], учителя [5, 3], $\tau = 0.99$, $lr=1$

Градиент ученика [-0.1, -0.2].

Веса ученика после шага: [0.9, 1.8]

Веса учителя после шага:

$0.99*[5, 3] + 0.01*[0.9, 1.8] = [4.95, 2.97] + [0.009, 0.018] = [4.969, 2.988]$

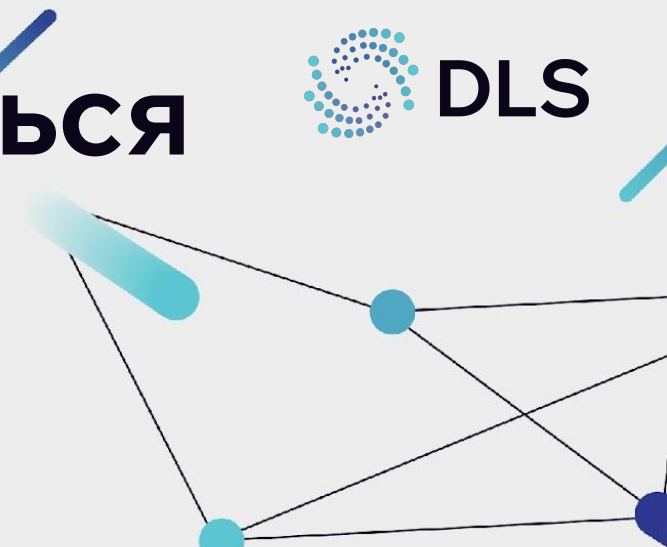
Апдейт учителя: [-0.031, -0.012]

Векторы [-0.1, -0.2] и [-0.031, -0.012] имеют разное направление -> учитель и ученик учатся разным вещам

Ученик шагает
куда
указывают
градиенты
батча

$$\theta \leftarrow \text{optimizer}(\theta, \nabla_{\theta} \mathcal{L}_{\theta, \xi}^{\text{BYOL}}, \eta),$$
$$\xi \leftarrow \tau \xi + (1 - \tau) \theta,$$

Учитель
шагает в
сторону ближе
к ученику



BYOL: почему это хорошо работает?

Откуда в таргете от учителя информация на которой может учиться ученик?

Почему это работает даже если мы учимся на выдаче случайно инициализированной модели?

Главное:

Голова-предиктор стремится воспроизвести **конкретную репрезентацию** на основании **конкретных входных данных**.

Method	Predictor	Target network	β	Top-1
BYOL	✓	✓	0	72.5
—	✓	✓	1	70.9
—		✓	1	70.7
SimCLR			1	69.4
—	✓		1	69.1
—	✓		0	0.3
—		✓	0	0.2
—			0	0.1

(b) Intermediate variants between BYOL and SimCLR.

BYOL: разгадка

Коллапса не происходит из-за противоборства между MLP-предиктором и учителем.

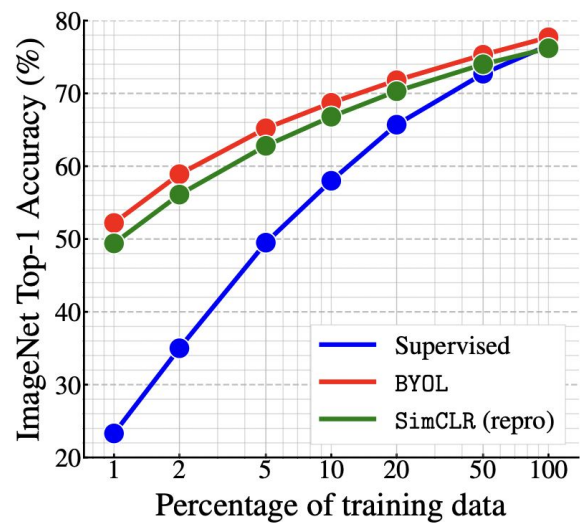
1. Предсказания ученика всегда отличаются из-за наличия предиктора.
2. Учитель со скользящим средним дает более стабильные предсказания.
3. Обновление учителя происходит не в ту сторону, что обновление ученика.

Голова-предиктор стремится воспроизвести **конкретную репрезентацию** (даже если её выдала случайная модель) на основании **конкретных входных** данных.

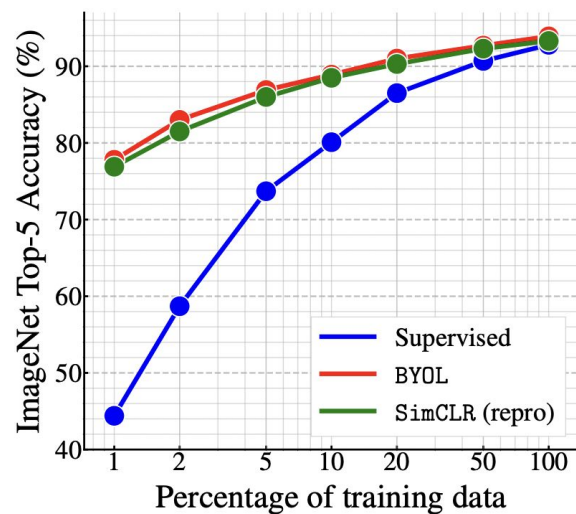
Method	Predictor	Target network	β	Top-1
BYOL	✓	✓	0	72.5
—	✓	✓	1	70.9
—		✓	1	70.7
SimCLR			1	69.4
—	✓		1	69.1
—	✓		0	0.3
—		✓	0	0.2
—			0	0.1

(b) Intermediate variants between BYOL and SimCLR.

BYOL: результаты

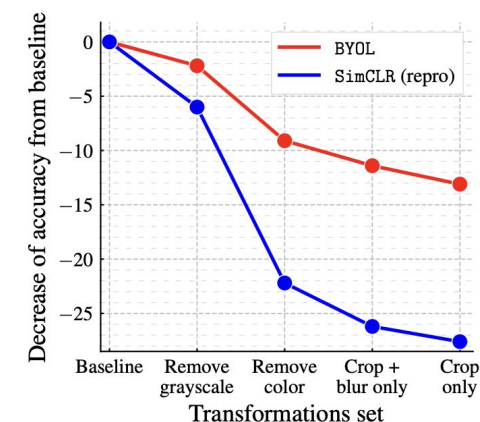
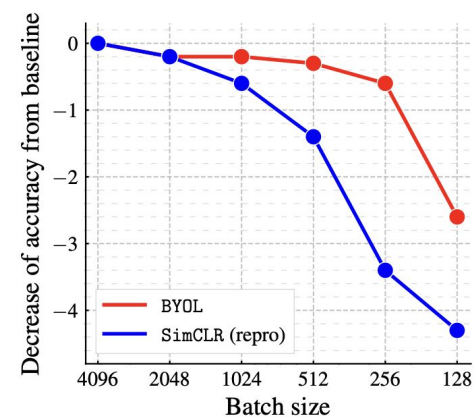


(a) Top-1 accuracy



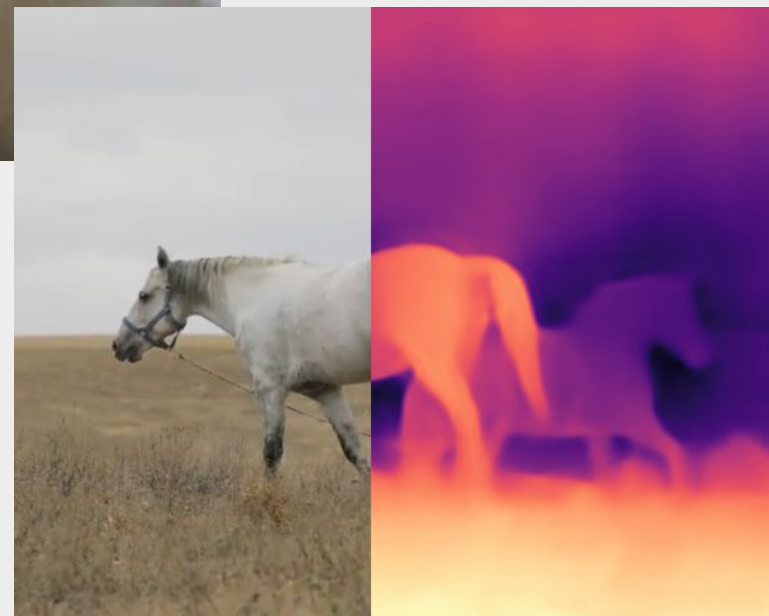
(b) Top-5 accuracy

Figure 4: Semi-supervised training with a fraction of ImageNet labels on a ResNet-50 ($\times 1$).



Проблемы VYOL

- Гиперпараметры.
- Просто хочется лучше.
- Делает векторы, а мы хотим делать более сложные вещи: object detection, depth estimation, итд.





Борис Цейтлин

Админ @boris_again, ex-Staff MLE eBay

Self-Supervised Computer Vision

Часть 2

Современные SSL методы в CV: Dino (v2, v3) и MAE

DINOv1: Distillation with NO labels

Emerging Properties in Self-Supervised Vision Transformers

Mathilde Caron^{1,2} Hugo Touvron^{1,3} Ishan Misra¹ Hervé Jegou¹
 Julien Mairal² Piotr Bojanowski¹ Armand Joulin¹

¹ Facebook AI Research ² Inria* ³ Sorbonne University

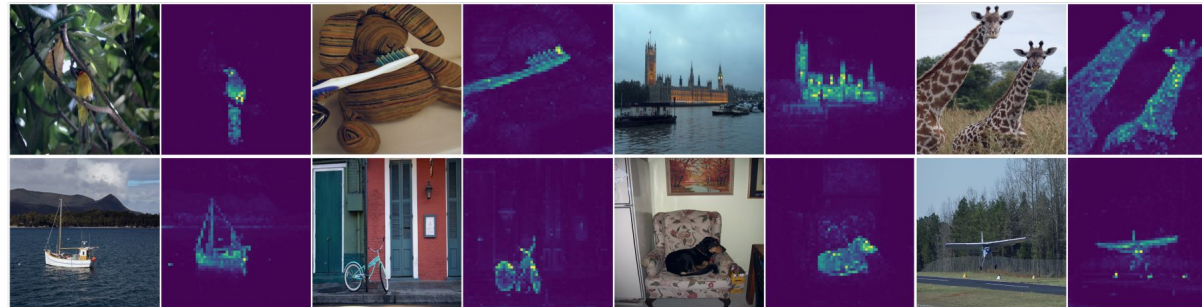


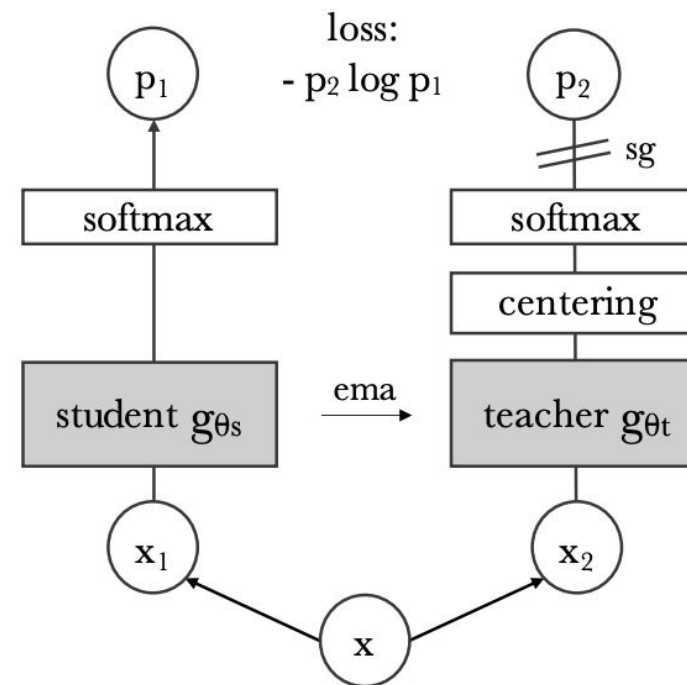
Figure 1: **Self-attention from a Vision Transformer with 8×8 patches trained with no supervision.** We look at the self-attention of the [CLS] token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

<https://arxiv.org/pdf/2104.14294>

DINOv1: Метод

Отличия от BYOL:

- Лосс это кросс-энтропия
- Global and local views
 - Учитель видит только global views
- Cosine schedule для EMA учителя
- Нет MLP predictor, только projection head.
 - Но projection head имеет 3 скрытых слоя.
- Centering, sharpening



$$\min_{\theta_s} \sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in V \\ x' \neq x}} H(P_t(x), P_s(x')). \quad (3)$$

DINOv1: projection head

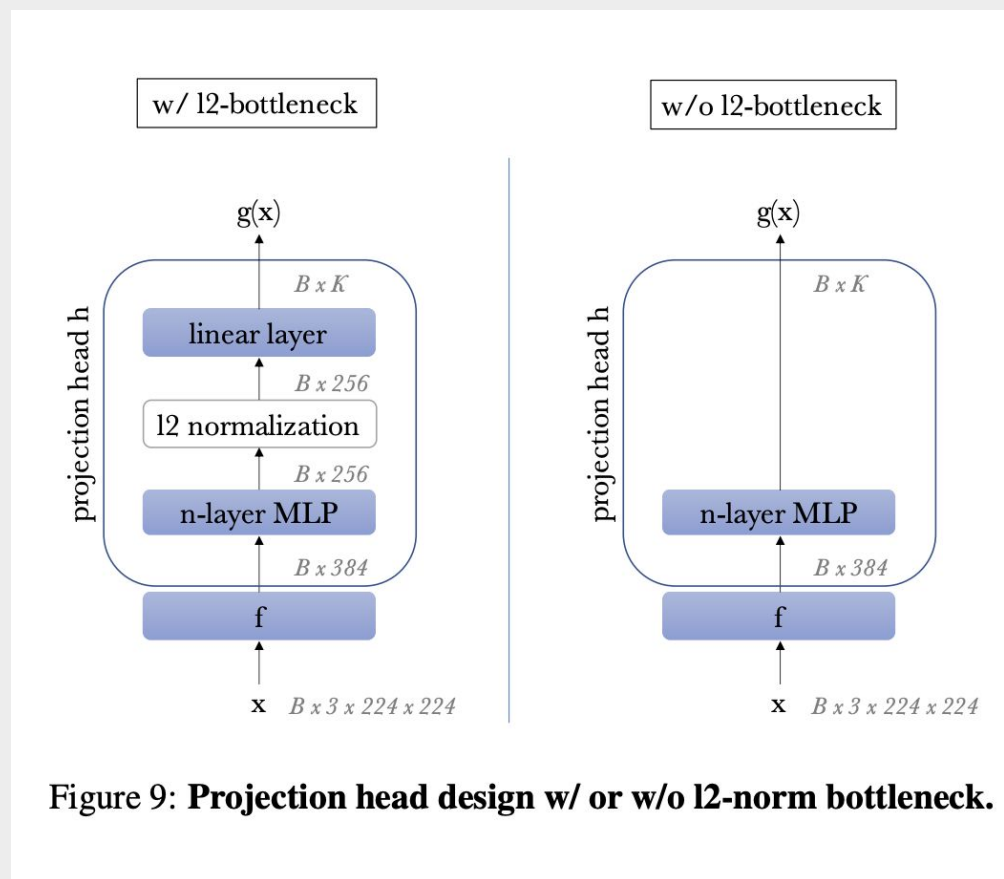
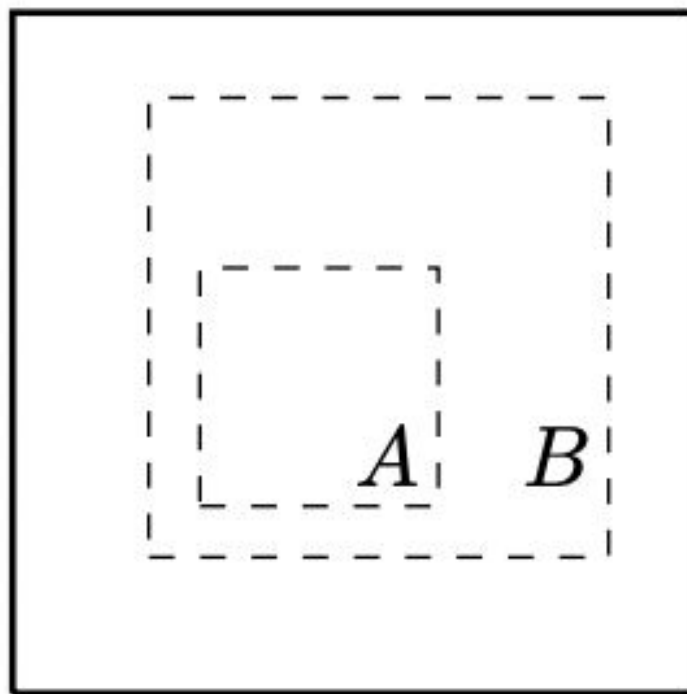
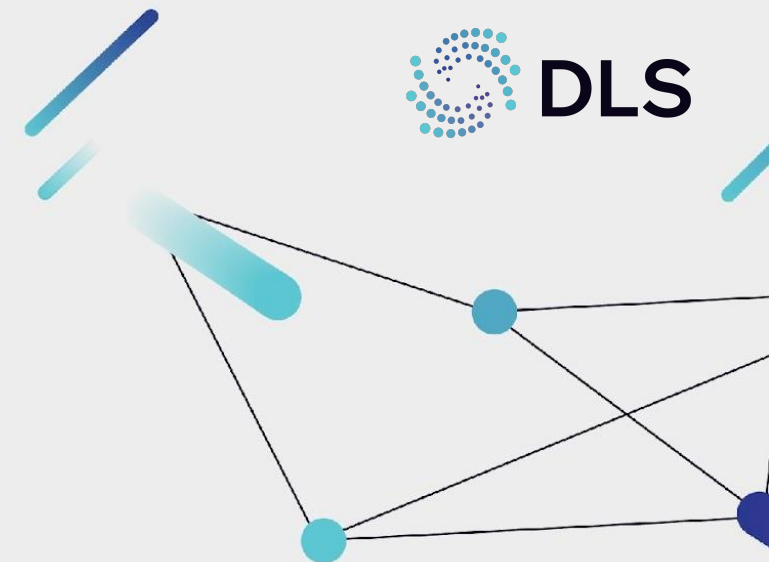


Figure 9: Projection head design w/ or w/o l2-norm bottleneck.

DINOv1: Local/Global Recap



(a) Global and local views.



DINOv1: Алгоритм

- Много local random views для ученика
- Center – Вектор констант
- Centering: (t - C)
 - Bias term для учителя
 - Толкает предсказания к равномерному распределению
- Sharpening: деление на температуру
 - Толкает предсказания к one-hot распределению

Вместе centering и sharpening тянут в разные стороны и помогают предотвратить коллапс.

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

$g_t(x) \leftarrow g_t(\bar{x}) + c$. The center c is updated with an exponential moving average, which allows the approach to work well across different batch sizes as shown in Section 5.5:

$$c \leftarrow mc + (1 - m) \frac{1}{B} \sum_{i=1}^B g_{\theta_t}(x_i), \quad (4)$$

where $m > 0$ is a rate parameter and B is the batch size. Output sharpening is obtained by using a low value for the temperature τ_t in the teacher softmax normalization.

DINOv1: Лосс

$$\min_{\theta_s} \sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in V \\ x' \neq x}} H(P_t(x), P_s(x')). \quad (3)$$

global
views

все
остальные
кропы

предсказание
учителя для
глобального
кропа

предсказание
студента для
глобального
или локального
кропа

В случае ViT: рассчитывается только
на CLS токенах!

DINOv1: Хорошие фишки

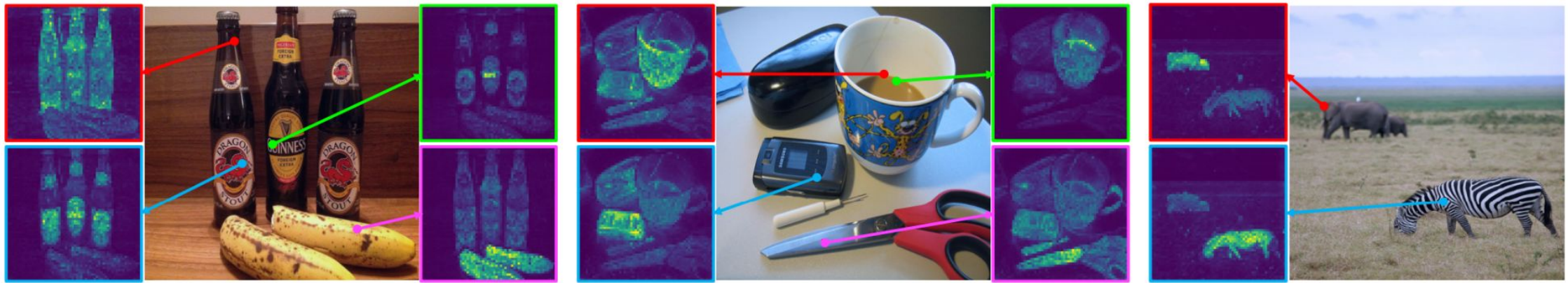


Figure 8: **Self-attention for a set of reference points.** We visualize the self-attention module from the last block of a ViT-S/8 trained with DINO. The network is able to separate objects, though it has been trained with no supervision at all.

DINOv1: Хорошие фишки

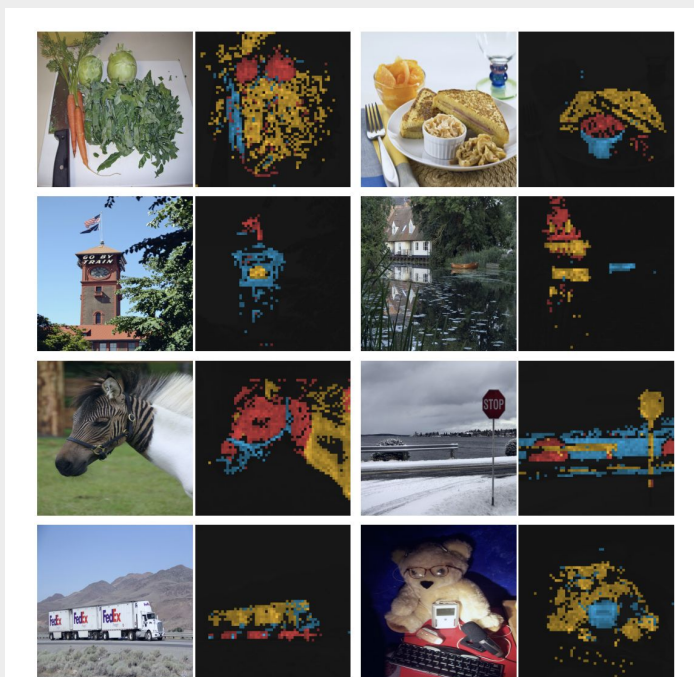
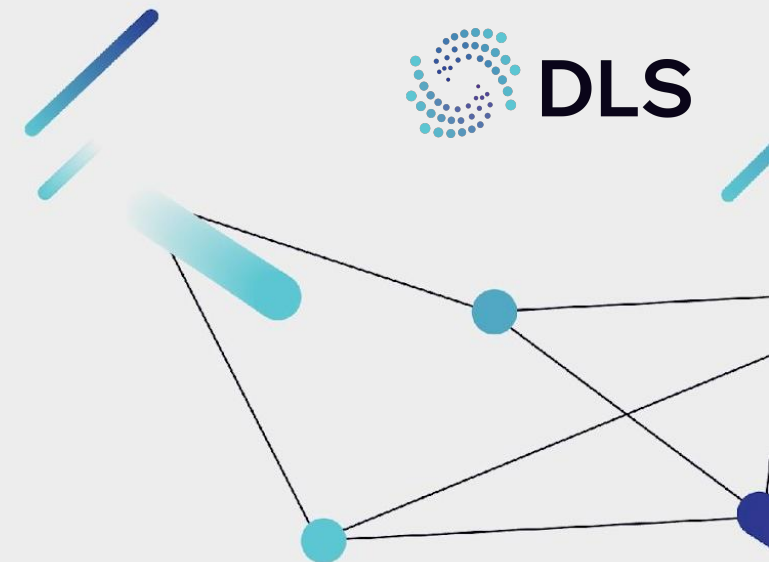


Figure 3: **Attention maps from multiple heads.** We consider the heads from the last layer of a ViT-S/8 trained with DINO and display the self-attention for [CLS] token query. Different heads, materialized by different colors, focus on different locations that represents different objects or parts (more examples in Appendix).



DINOv1: SSL сегментация из коробки

Supervised



DINO



	Random	Supervised	DINO
ViT-S/16	22.0	27.3	45.9
ViT-S/8	21.8	23.7	44.7

Figure 4: **Segmentations from supervised versus DINO.** We visualize masks obtained by thresholding the self-attention maps to keep 60% of the mass. On top, we show the resulting masks for a ViT-S/8 trained with supervision and DINO. We show the best head for both models. The table at the bottom compares the Jac-card similarity between the ground truth and these masks on the validation images of PASCAL VOC12 dataset.

DINOv2

Published in Transactions on Machine Learning Research (01/2024)

DINOv2: Learning Robust Visual Features without Supervision

Maxime Oquab**, Timothée Darcet**, Théo Moutakanni**,
Huy V. Vo*, Marc Szafraniec*, Vasil Khalidov*, Pierre Fernandez, Daniel Haziza,
Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba,
Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat,
Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal¹,
Patrick Labatut*, Armand Joulin*, Piotr Bojanowski*

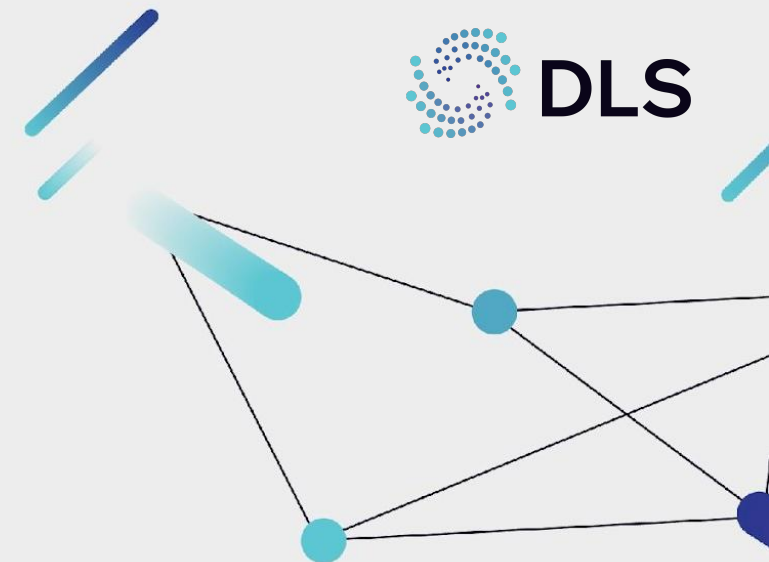
Meta AI Research ¹*Inria*

*core team **equal contribution

Reviewed on OpenReview: <https://openreview.net/forum?id=a68SUt6zFt>

<https://arxiv.org/pdf/2304.07193.pdf>

DINOv2



DINOv2

- **Image-level objective (Caron et al., 2021).** We consider the cross-entropy loss between the features extracted from a student and a teacher network. Both features are coming from the class token of a ViT, obtained from different crops of the same image. We pass the student class token through the student DINO head. This head is an MLP model outputting a vector of scores, that we call "prototype scores". We then apply a softmax to obtain p_s . Similarly, we apply the teacher DINO head to the teacher class token to obtain teacher prototype scores. We then apply a softmax followed by a centering with moving average (or a Sinkhorn-Knopp centering as detailed thereafter) to obtain p_t . The DINO loss term corresponds to:

$$\mathcal{L}_{DINO} = - \sum p_t \log p_s$$

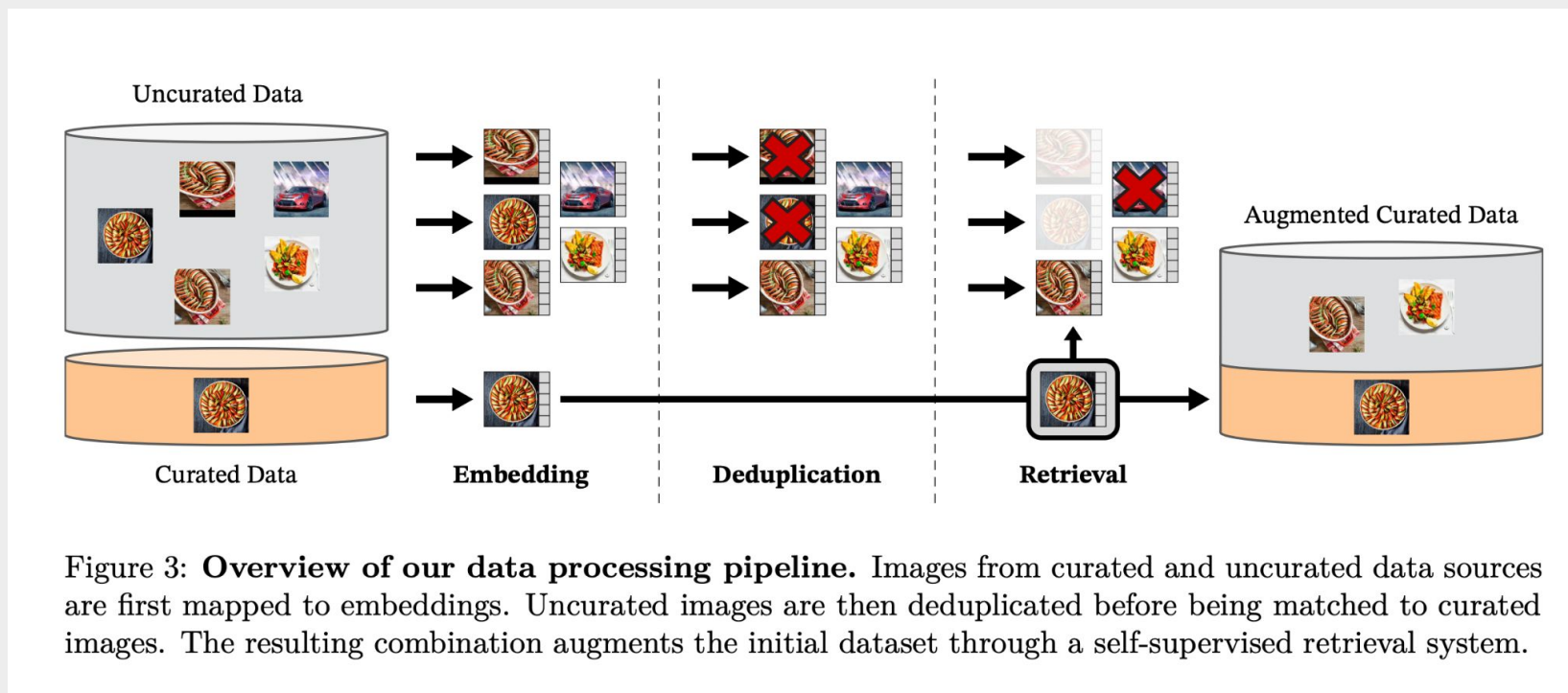
We learn the parameters of the student and build the teacher head with an exponential moving average of past iterates (He et al., 2020).

- **Patch-level objective (Zhou et al., 2022a).** We randomly mask some of the input patches given to the student, but not to the teacher. We then apply the student iBOT head to the student mask tokens. Similarly, we apply the teacher iBOT head to the (visible) teacher patch tokens corresponding to the ones masked in the student. We then apply the softmax and centering steps as above, and obtain the iBOT loss term:

$$\mathcal{L}_{iBOT} = - \sum_i p_{ti} \log p_{si}$$

DINOv2

Датасет: 142М изображений



DINOv2: ещё более хорошие многоканальные признаки

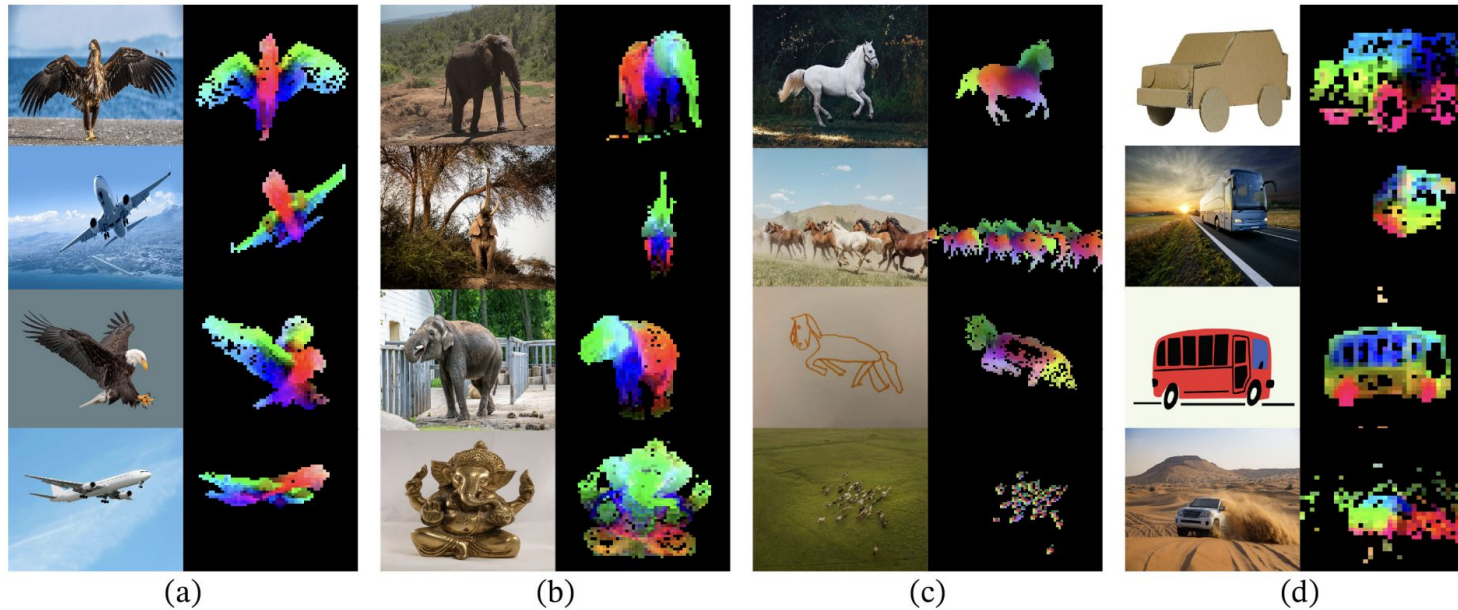


Figure 1: **Visualization of the first PCA components.** We compute a PCA between the patches of the images from the same column (a, b, c and d) and show their first 3 components. Each component is matched to a different color channel. Same parts are matched between related images despite changes of pose, style or even objects. Background is removed by thresholding the first PCA component.

DINOv2: результаты

Method	Arch.	Data	Text sup.	kNN	linear		
				val	val	ReaL	V2
Weakly supervised							
CLIP	ViT-L/14	WIT-400M	✓	79.8	84.3	88.1	75.3
CLIP	ViT-L/14 ₃₃₆	WIT-400M	✓	80.5	85.3	88.8	75.8
SWAG	ViT-H/14	IG3.6B	✓	82.6	85.7	88.7	77.6
OpenCLIP	ViT-H/14	LAION-2B	✓	81.7	84.4	88.4	75.5
OpenCLIP	ViT-G/14	LAION-2B	✓	83.2	86.2	89.4	77.2
EVA-CLIP	ViT-g/14	custom*	✓	83.5	86.4	89.3	77.4
Self-supervised							
MAE	ViT-H/14	INet-1k	✗	49.4	76.6	83.3	64.8
DINO	ViT-S/8	INet-1k	✗	78.6	79.2	85.5	68.2
SEERv2	RG10B	IG2B	✗	—	79.8	—	—
MSN	ViT-L/7	INet-1k	✗	79.2	80.7	86.0	69.7
EsViT	Swin-B/W=14	INet-1k	✗	79.4	81.3	87.0	70.4
Mugs	ViT-L/16	INet-1k	✗	80.2	82.1	86.9	70.8
iBOT	ViT-L/16	INet-22k	✗	72.9	82.3	87.5	72.4
DINOv2	ViT-S/14	LVD-142M	✗	79.0	81.1	86.6	70.9
	ViT-B/14	LVD-142M	✗	82.1	84.5	88.3	75.1
	ViT-L/14	LVD-142M	✗	83.5	86.3	89.5	78.0
	ViT-g/14	LVD-142M	✗	83.5	86.5	89.6	78.4

Table 4: **Linear evaluation on ImageNet-1k of frozen pretrained features.** We report Top-1 accuracy on the validation set for publicly available models trained on public or private data, and with or without text supervision (text sup.). For reference, we also report the kNN performance on the validation set. We compare across any possible architectures (Arch.), at resolution 224×224 unless stated otherwise. The dataset used for training EVA-CLIP is a custom mixture, see paper for details (Fang et al., 2023).

<https://arxiv.org/pdf/2304.07193.pdf>

DINOv3

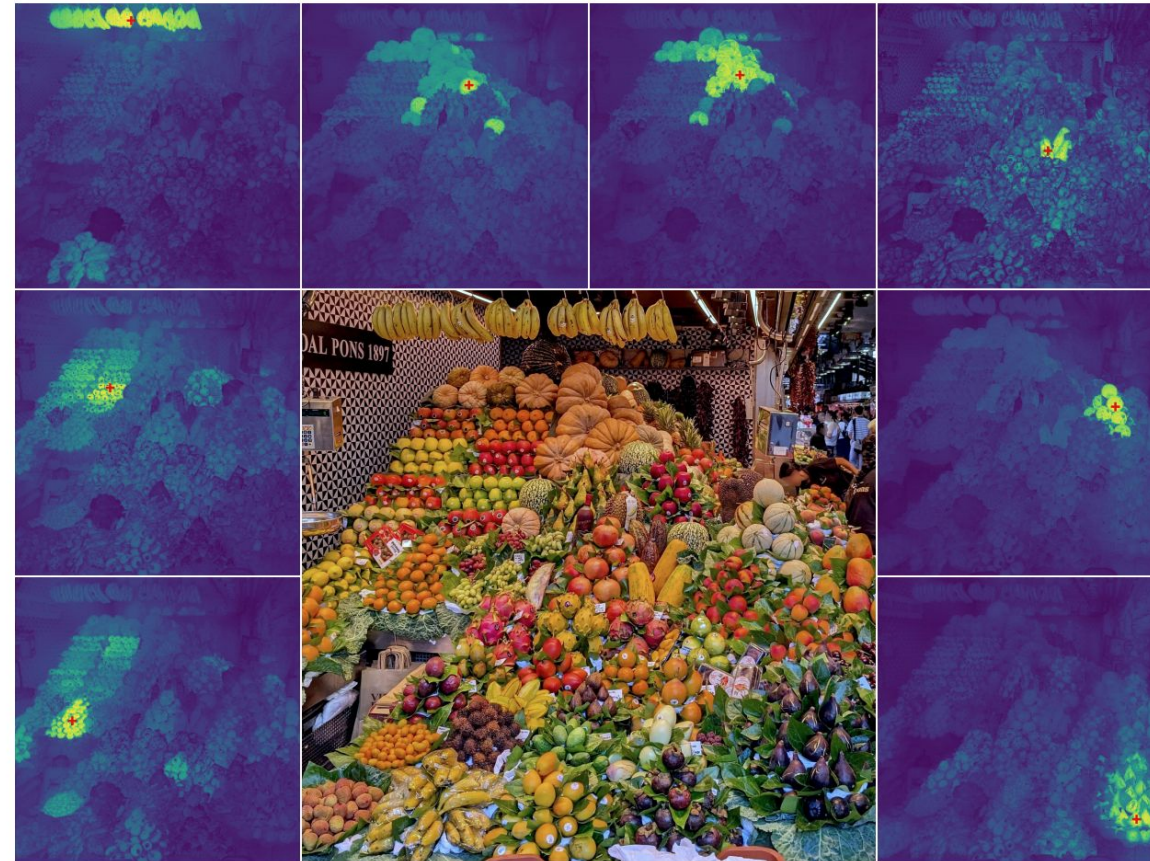


Figure 3: High-resolution dense features. We visualize the cosine similarity maps obtained with DINOv3 output features between the patches marked with a red cross and all other patches. Input image at 4096×4096 . Please zoom in, do you agree with DINOv3?

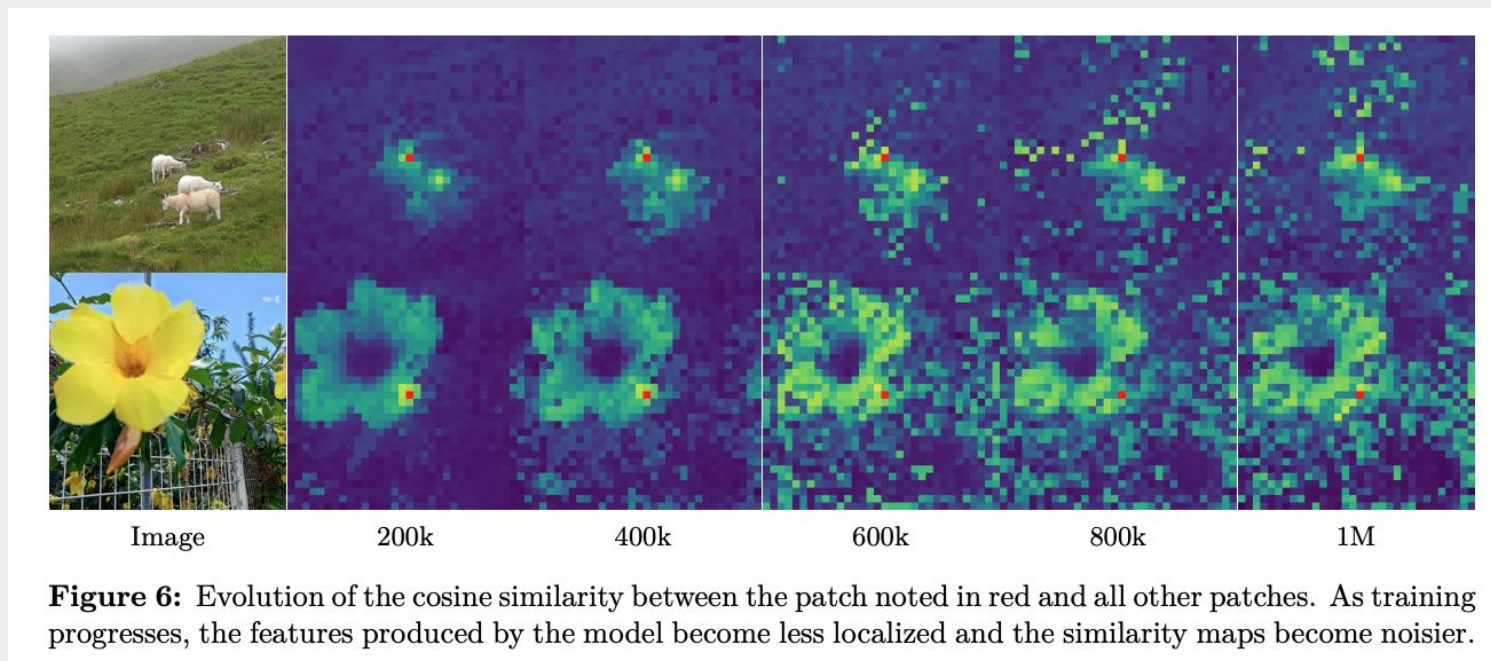
<https://arxiv.org/pdf/2508.10104>

DINOv3

Table 2: Comparison of the teacher architectures used in DINOv2 and DINOv3 models. We keep the model 40 blocks deep, and increase the embedding dimension to 4096. Importantly, we use a patch size of 16 pixels, changing the effective sequence length for a given resolution.

Teacher model	DINOv2	DINOv3
Backbone	ViT-giant	ViT-7B
#Params	1.1B	6.7B
#Blocks	40	40
Patch Size	14	16
Pos. Embeddings	Learnable	RoPE
Registers	4	4
Embed. Dim.	1536	4096
FFN Type	SwiGLU	SwiGLU
FFN Hidden Dim.	4096	8192
Attn. Heads	24	32
Attn. Heads Dim.	64	128
DINO Head MLP	4096-4096-256	8192-8192-512
DINO Prototypes	128k	256k
iBOT Head MLP	4096-4096-256	8192-8192-384
iBOT Prototypes	128k	96k

DINOv3



Проблема DINOv2: глобальный лосс со временем начинает доминировать

<https://arxiv.org/pdf/2508.10104>

DINOv3: Gram Anchoring

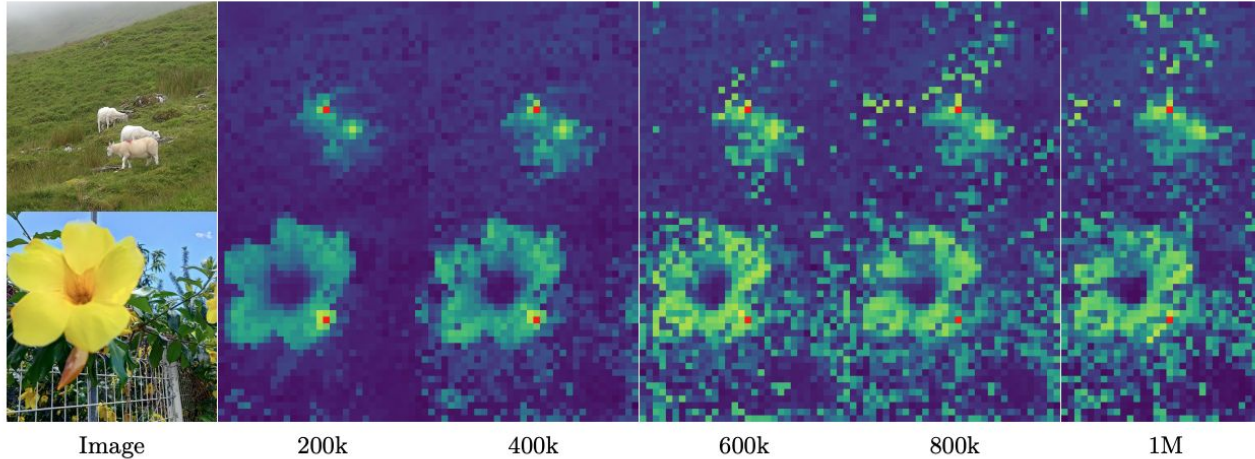


Figure 6: Evolution of the cosine similarity between the patch noted in red and all other patches. As training progresses, the features produced by the model become less localized and the similarity maps become noisier.

$$\mathcal{L}_{\text{Gram}} = \left\| \mathbf{X}_S \cdot \mathbf{X}_S^\top - \mathbf{X}_G \cdot \mathbf{X}_G^\top \right\|_F^2.$$

Masked Autoencoders Are Scalable Vision Learners

Kaiming He^{*,†} Xinlei Chen^{*} Saining Xie Yanghao Li Piotr Dollár Ross Girshick

^{*}equal technical contribution [†]project lead

Facebook AI Research (FAIR)

<https://arxiv.org/abs/2111.06377>

MAE: метод

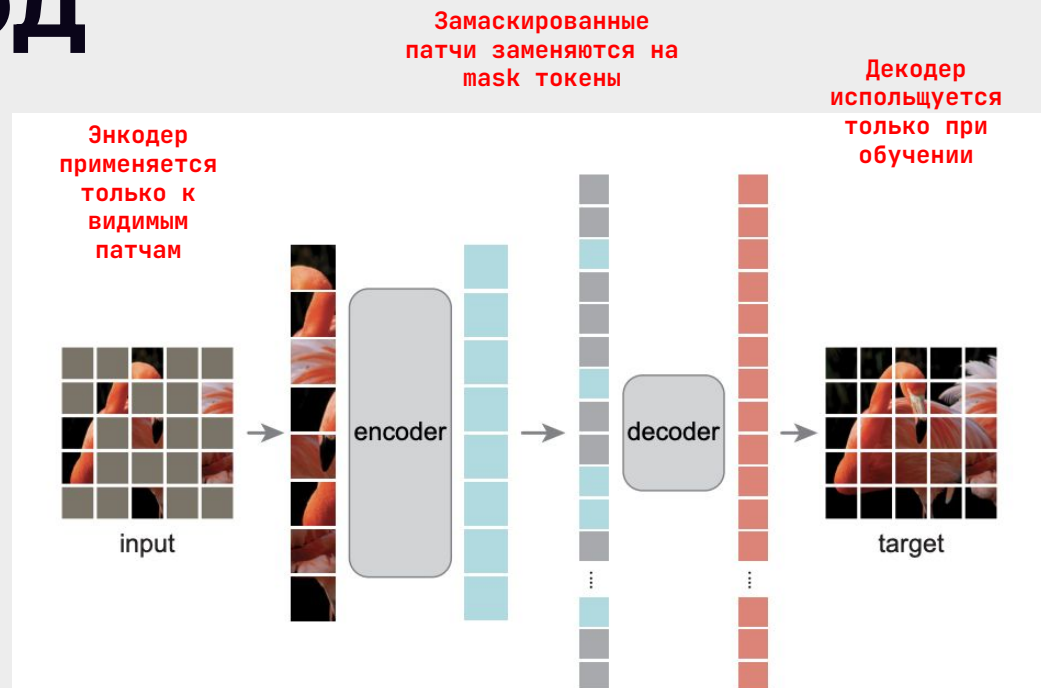


Figure 1. **Our MAE architecture.** During pre-training, a large random subset of image patches (*e.g.*, 75%) is masked out. The encoder is applied to the small subset of *visible patches*. Mask tokens are introduced *after* the encoder, and the full set of encoded patches and mask tokens is processed by a small decoder that reconstructs the original image in pixels. After pre-training, the decoder is discarded and the encoder is applied to uncorrupted images (full sets of patches) for recognition tasks.

MAE: метод

- Патчи выбираются равномерно случайно
- 80% патчей маскируются
 - Задача не решается простой интерполяцией
 - Так же это позволяет сделать эффективный энкодер

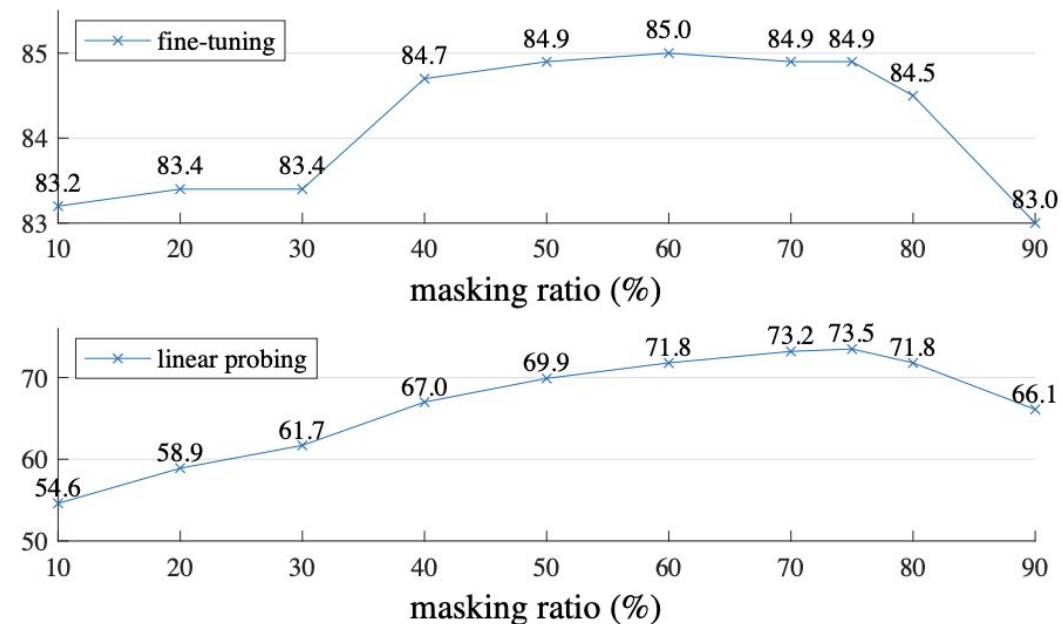


Figure 5. **Masking ratio.** A high masking ratio (75%) works well for both fine-tuning (top) and linear probing (bottom). The y-axes are ImageNet-1K validation accuracy (%) in all plots in this paper.

MAE: примеры

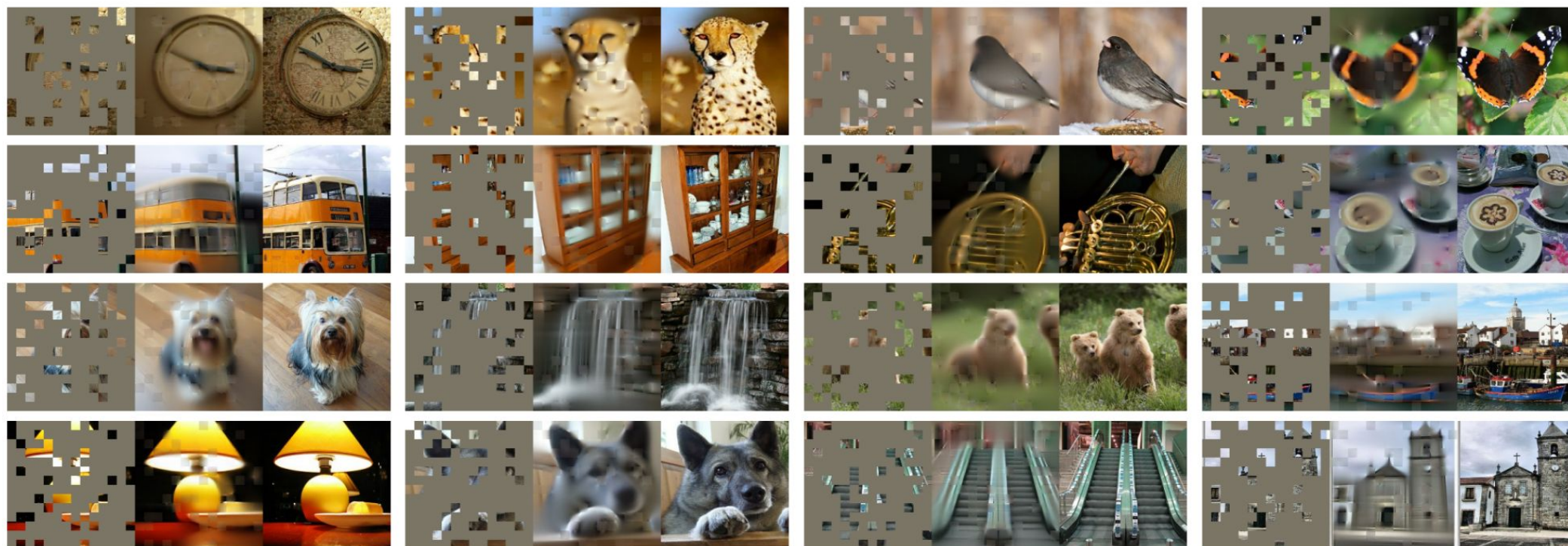


Figure 2. Example results on ImageNet *validation* images. For each triplet, we show the masked image (left), our MAE reconstruction[†] (middle), and the ground-truth (right). The masking ratio is 80%, leaving only 39 out of 196 patches. More examples are in the appendix.

[†]As no loss is computed on visible patches, the model output on visible patches is qualitatively worse. One can simply overlay the output with the visible patches to improve visual quality. We intentionally opt not to do this, so we can more comprehensively demonstrate the method's behavior.

MAE: наблюдается генерализация

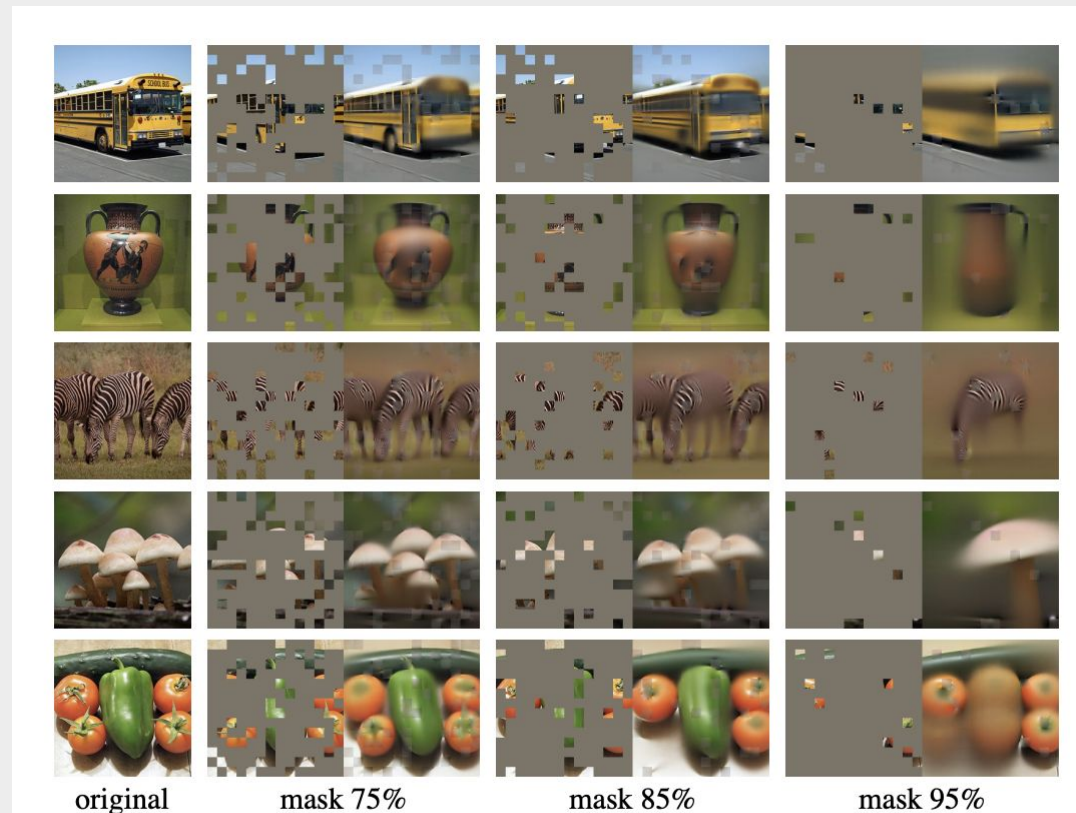
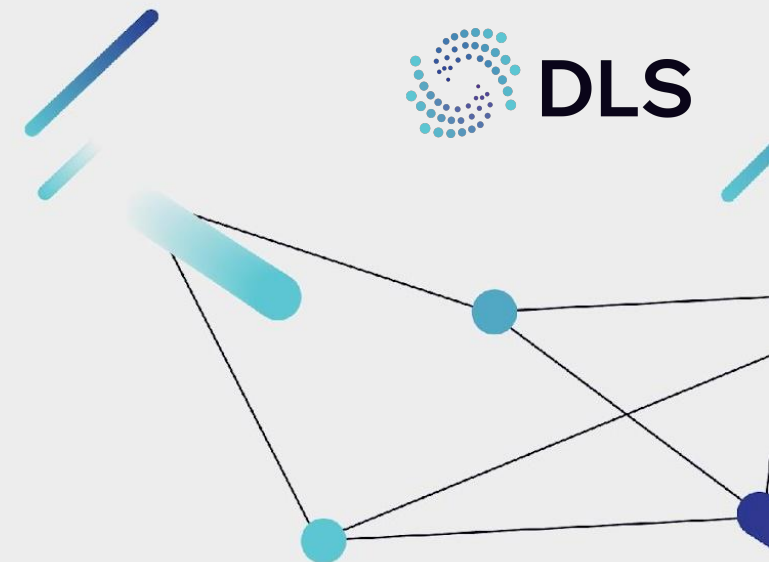


Figure 4. Reconstructions of ImageNet *validation* images using an MAE pre-trained with a masking ratio of 75% but applied on inputs with higher masking ratios. The predictions differ plausibly from the original images, showing that the method can generalize.

MAE: pixel-wise MSE loss

$$L = \|y_i - f(x)_i\|$$



MAE: маленький декодер, абляции

blocks	ft	lin
1	84.8	65.5
2	84.9	70.0
4	84.9	71.9
8	84.9	73.5
12	84.4	73.3

(a) **Decoder depth.** A deep decoder can improve linear probing accuracy.

case	ft	lin
pixel (w/o norm)	84.9	73.5
pixel (w/ norm)	85.4	73.9
PCA	84.6	72.3
dVAE token	85.3	71.6

(d) **Reconstruction target.** Pixels as reconstruction targets are effective.

dim	ft	lin
128	84.9	69.1
256	84.8	71.3
512	84.9	73.5
768	84.4	73.1
1024	84.3	73.1

(b) **Decoder width.** The decoder can be narrower than the encoder (1024-d).

case	ft	lin
none	84.0	65.7
crop, fixed size	84.7	73.1
crop, rand size	84.9	73.5
crop + color jit	84.3	71.9

(e) **Data augmentation.** Our MAE works with minimal or no augmentation.

MAE: стратегия маскинга

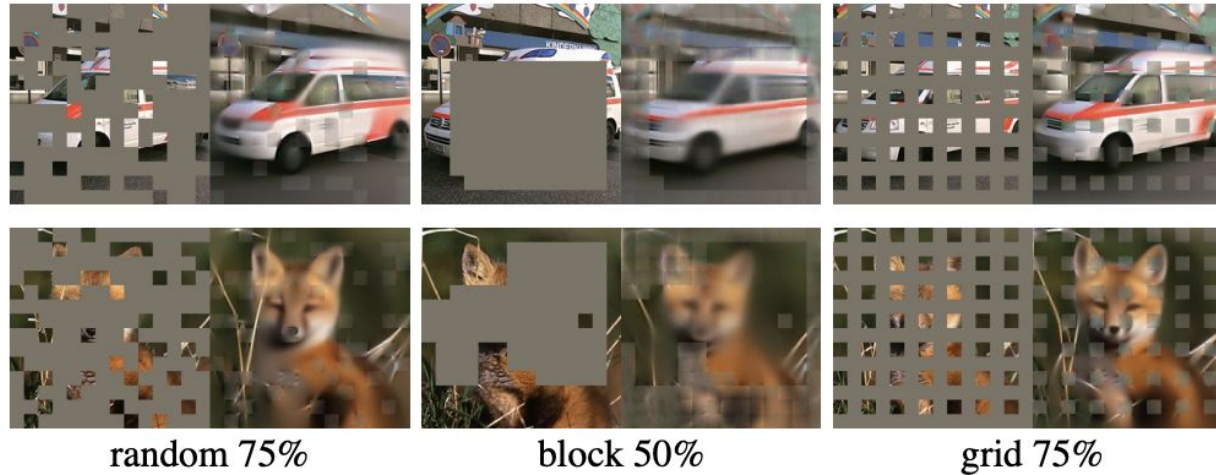
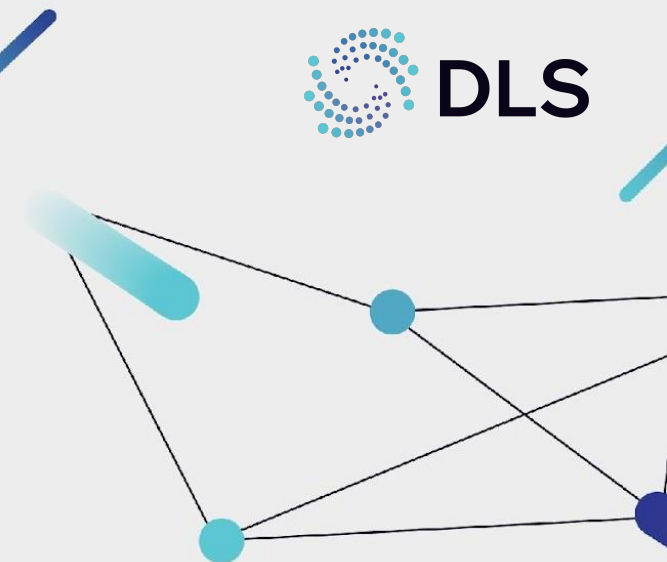


Figure 6. **Mask sampling strategies** determine the pretext task difficulty, influencing reconstruction quality and representations (Table 1f). Here each output is from an MAE trained with the specified masking strategy. Left: random sampling (our default). Middle: block-wise sampling [2] that removes large random blocks. Right: grid-wise sampling that keeps one of every four patches. Images are from the validation set.

MAE: предсказание пикселей сработало лучше токенов

We also compare an MAE variant that predicts *tokens*, the target used in BEiT [2]. Specifically for this variant, we use the DALLE pre-trained dVAE [50] as the tokenizer, following [2]. Here the MAE decoder predicts the token indices using cross-entropy loss. This tokenization improves fine-tuning accuracy by 0.4% *vs.* unnormalized pixels, but has no advantage *vs.* normalized pixels. It also reduces linear probing accuracy. In §5 we further show that tokenization is not necessary in transfer learning.

Our *pixel*-based MAE is much simpler than tokenization. The dVAE tokenizer requires one more pre-training stage, which may depend on extra data (250M images [50]). The dVAE encoder is a large convolutional network (40% FLOPs of ViT-L) and adds nontrivial overhead. Using pixels does not suffer from these problems.



MAE: сколько это учить

encoder	dec. depth	ft acc	hours	speedup
ViT-L, w/ [M]	8	84.2	42.4	-
ViT-L	8	84.9	15.4	2.8×
ViT-L	1	84.8	11.6	3.7×
ViT-H, w/ [M]	8	-	119.6 [†]	-
ViT-H	8	85.8	34.5	3.5×
ViT-H	1	85.9	29.3	4.1×

Table 2. **Wall-clock time** of our MAE training (800 epochs), benchmarked in 128 TPU-v3 cores with TensorFlow. The speedup is relative to the entry whose encoder has mask tokens (gray). The decoder width is 512, and the mask ratio is 75%. [†]: This entry is estimated by training ten epochs.

config	value
optimizer	AdamW [39]
base learning rate	1.5e-4
weight decay	0.05
optimizer momentum	$\beta_1, \beta_2=0.9, 0.95$ [6]
batch size	4096
learning rate schedule	cosine decay [38]
warmup epochs [20]	40
augmentation	RandomResizedCrop

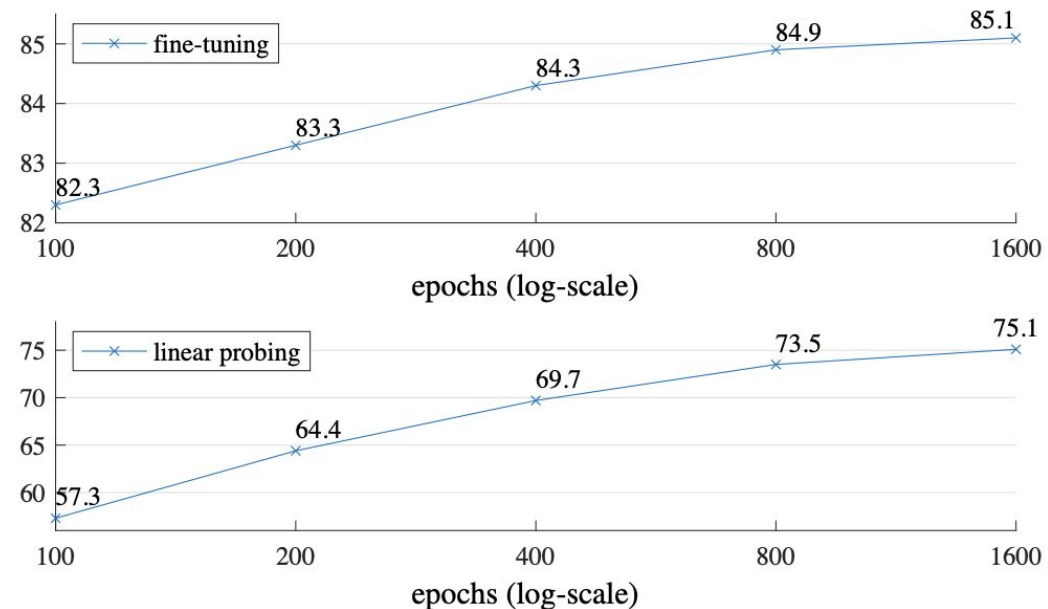


Figure 7. **Training schedules.** A longer training schedule gives a noticeable improvement. Here each point is a full training schedule. The model is ViT-L with the default setting in Table 1.

MAE: результаты

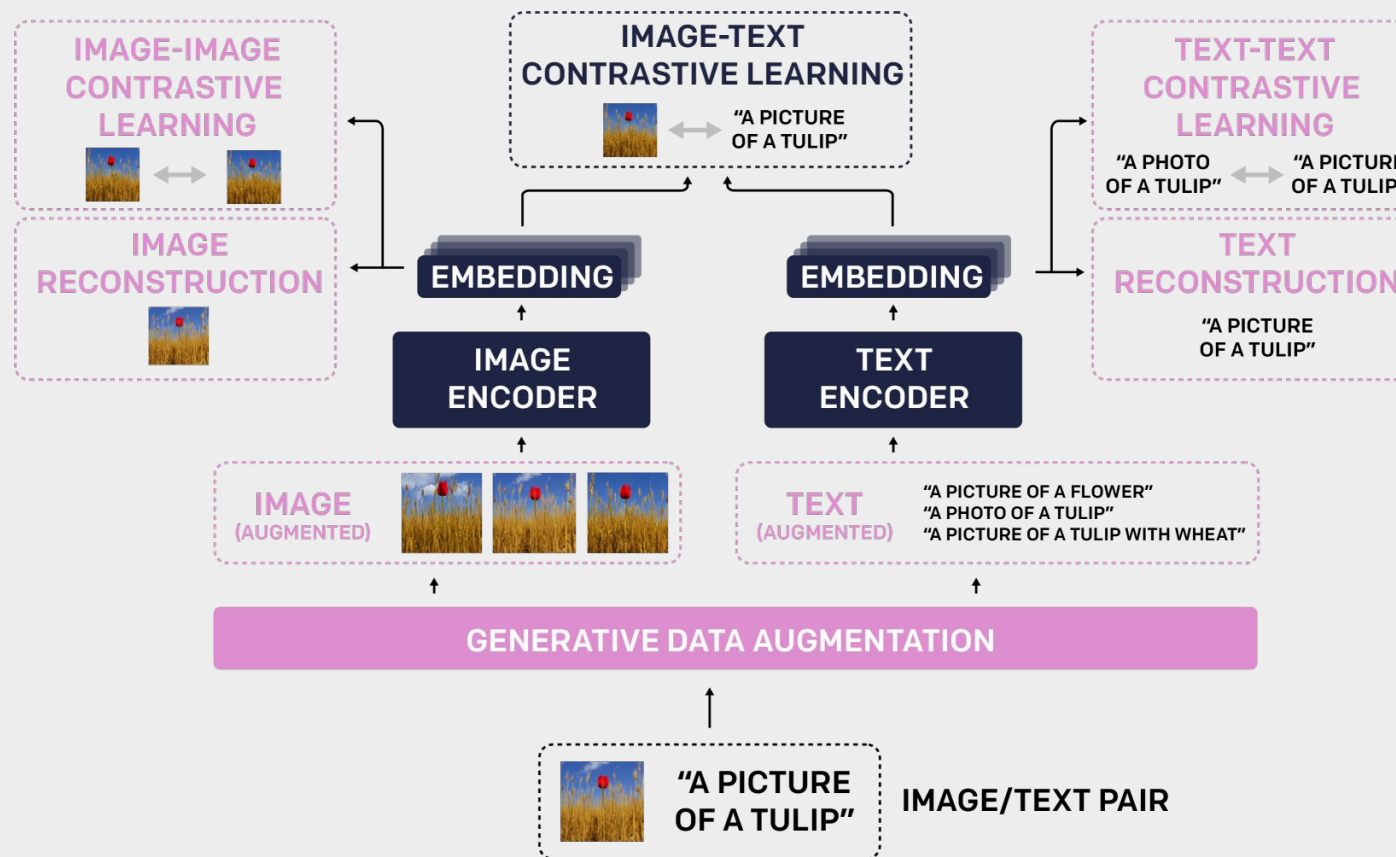
method	pre-train data	ViT-B	ViT-L	ViT-H	ViT-H ₄₄₈
scratch, our impl.	-	82.3	82.6	83.1	-
DINO [5]	IN1K	82.8	-	-	-
MoCo v3 [9]	IN1K	83.2	84.1	-	-
BEiT [2]	IN1K+DALLE	83.2	85.2	-	-
MAE	IN1K	<u>83.6</u>	<u>85.9</u>	<u>86.9</u>	87.8

Table 3. **Comparisons with previous results on ImageNet-1K.** The pre-training data is the ImageNet-1K training set (except the tokenizer in BEiT was pre-trained on 250M DALLE data [50]). All self-supervised methods are evaluated by end-to-end fine-tuning. The ViT models are B/16, L/16, H/14 [16]. The best for each column is underlined. All results are on an image size of 224, except for ViT-H with an extra result on 448. Here our MAE reconstructs normalized pixels and is pre-trained for 1600 epochs.

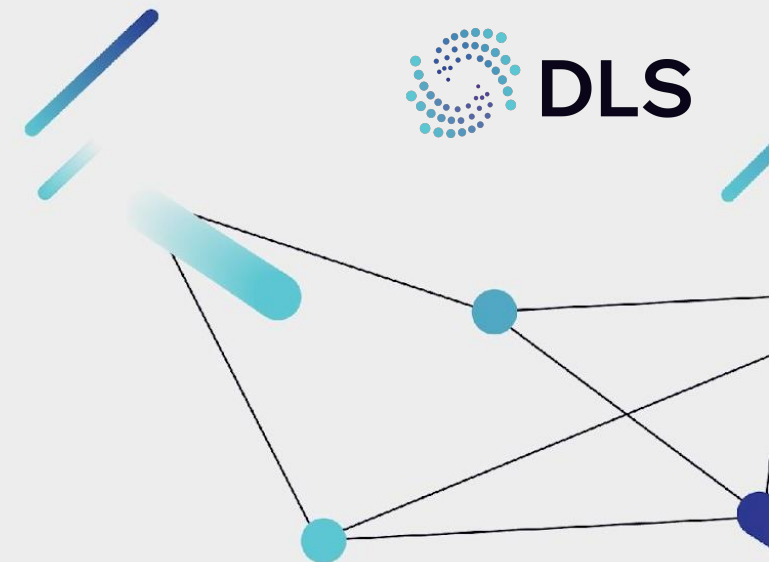
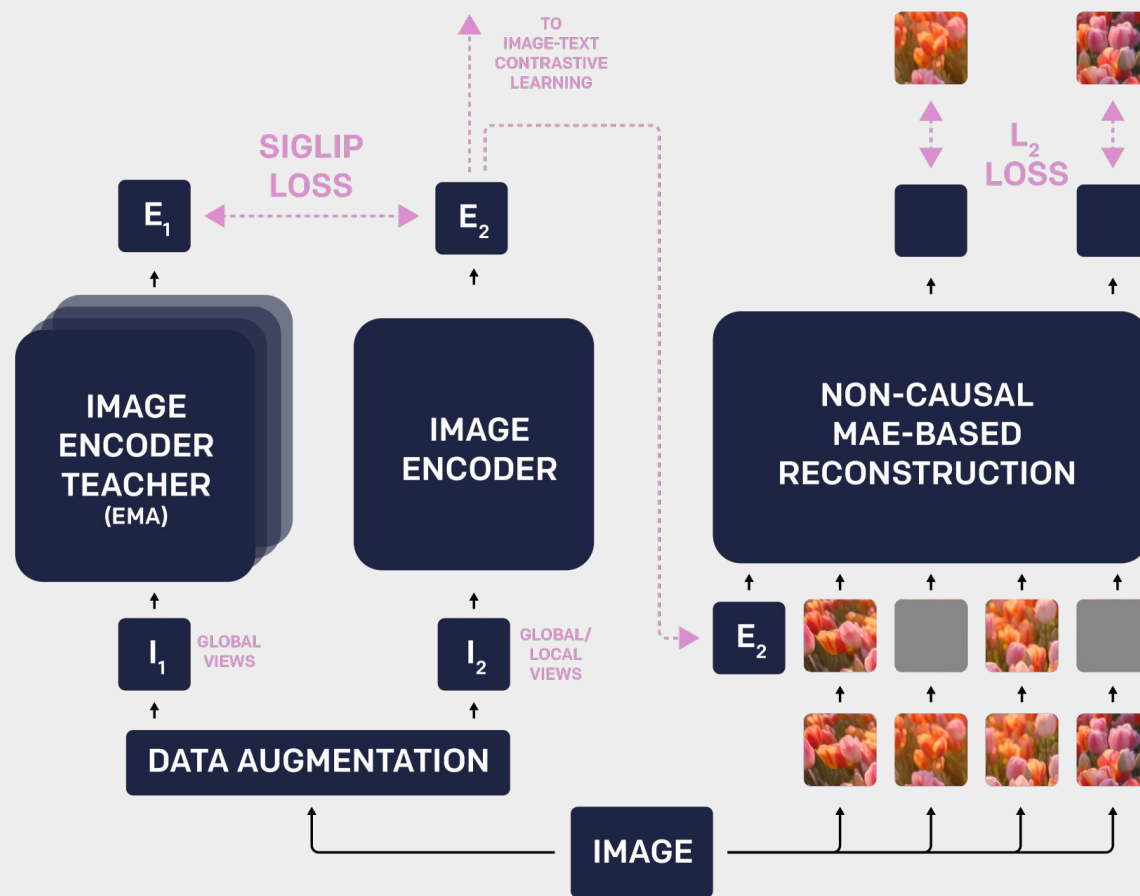
TULIP: Как эти методы стали частью VLM

<https://arxiv.org/html/2503.15485v2>

TULIP: CLIP + SSL методы



TULIP: Image Encoder





Борис Цейтлин

ex-Staff MLE eBay

Спасибо за внимание!

Канал: @boris_again

А ещё я пишу книгу про ML для
чайников